
Fundamentos de cifrado y autenticación

PID_00289975

Xavier Perramon Tornil

Tiempo mínimo de dedicación recomendado: 4 horas



**Xavier Perramon Tornil**

Doctor ingeniero de Telecomunicación por la Universidad Politécnica de Cataluña (UPC). Ha trabajado como consultor de seguridad en sistemas TIC. Actualmente es profesor de Sistemas de Información en la Escuela Superior de Ingenierías Industrial, Aeroespacial y Audiovisual de Terrassa (ESEIAAT) de la UPC.

La revisión de este recurso de aprendizaje UOC ha sido coordinada por el profesor: Victor Garcia Font

Sexta edición: septiembre 2022
© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)
Av. Tibidabo, 39-43, 08035 Barcelona
Autoría: Xavier Perramon Tornil
Producción: FUOC

Se garantiza permiso para copiar, distribuir y modificar este documento según los términos de la GNU Free Documentation License, Version 1.3 o cualquiera posterior publicada por la Free Software Foundation, sin secciones invariantes ni textos de cubierta delantera o trasera. Pueden consultarse los términos de la licencia en <http://www.gnu.org/licenses/fdl-1.3.html>.

Índice general

Introducción	5
Objetivos	6
1. Conceptos básicos de criptografía	7
1.1. Criptografía de clave simétrica	9
1.1.1. Algoritmos de cifrado en flujo	10
1.1.2. Algoritmos de cifrado en bloque	13
1.1.3. Uso de los algoritmos de clave simétrica	15
1.1.4. Funciones <i>hash</i> seguras	18
1.2. Criptografía de clave pública	20
1.2.1. Algoritmos de clave pública	20
1.2.2. Uso de la criptografía de clave pública	23
1.3. Infraestructura de clave pública (PKI)	25
1.3.1. Certificados de clave pública	25
1.3.2. Cadenas de certificados y jerarquías de certificación	28
1.3.3. Listas de revocación de certificados (CRL)	28
2. Sistemas de autenticación	30
2.1. Autenticación de mensaje	30
2.1.1. Códigos de autenticación de mensaje (MAC)	30
2.1.2. Firmas digitales	31
2.2. Autenticación de entidad	31
2.2.1. Contraseñas	32
2.2.2. Protocolos de reto-respuesta	40
Resumen	46
Glosario	48
Bibliografía	52

Introducción

Para proteger las redes de comunicaciones, la **criptografía** es la herramienta que nos permite evitar que alguien intercepte, manipule o falsifique los datos transmitidos. Dedicaremos la primera parte de este módulo a introducir los conceptos de criptografía necesarios para entender cómo se aplica a la protección de las comunicaciones.

La finalidad básica de la criptografía es el envío de información secreta. Si aplicamos una transformación, conocida como **cifrado**, a la información que queremos mantener en secreto, aunque un adversario consiga ver qué datos estamos enviando le serán completamente ininteligibles. Sólo el destinatario legítimo será capaz de realizar la transformación inversa y recuperar los datos originales.

Pero más allá de mantener la información en secreto, existen otros servicios que pueden ser igualmente necesarios, como, por ejemplo, la **autenticación**. Debemos evitar, de esta forma, que después de tomar todas las medidas necesarias para que sólo el destinatario final pueda leer la información, resulte que este destinatario sea un impostor que haya conseguido hacerse pasar por el auténtico destinatario. En la segunda parte del módulo veremos algunos sistemas para garantizar la autenticidad en las comunicaciones, la mayoría de ellos basados en técnicas criptográficas.

Objetivos

Los conceptos presentados en el presente módulo didáctico deben permitir al estudiante alcanzar los siguientes objetivos:

1. Saber qué funciones nos ofrece la criptografía, tanto las técnicas de clave simétrica como las de clave pública.
2. Conocer los distintos algoritmos de cifrado, integridad y autenticación disponibles y sus posibles usos.
3. Combinar las herramientas de criptografía pública con las de criptografía simétrica para conseguir distintas prestaciones.
4. Conocer el uso de los certificados X.509 y las listas de revocación, su estructura y la utilidad de los distintos campos.
5. Reconocer la necesidad de los sistemas de autenticación, qué técnicas concretas utilizan, y cómo estas técnicas permiten contrarrestar los intentos de suplantación.

1. Conceptos básicos de criptografía

A lo largo de la historia se han diseñado distintas técnicas para ocultar el significado de la información que no interesa que sea conocida por extraños. Algunas de ellas ya se usaban en tiempos de la antigua Grecia o del Imperio romano: por ejemplo, se atribuye a Julio César la invención de un código para enviar mensajes cifrados que no pudieran ser interpretados por el enemigo.

Criptografía

Los términos **criptografía**, **criptología**, etc. provienen de la raíz griega *kryptós*, que significa “escondido”.

La **criptografía** estudia, desde un punto de vista matemático, los métodos de protección de la información. Por otro lado, el **criptoanálisis** estudia las posibles técnicas utilizadas para contrarrestar los métodos criptográficos, y es de gran utilidad para ayudar a que estos sean más robustos y difíciles de atacar. El conjunto formado por estas dos disciplinas, criptografía y criptoanálisis, se conoce como **criptología**.

Cuando la protección que queremos obtener consiste en garantizar el secreto de la información, es decir, la **confidencialidad**, utilizamos el método criptográfico conocido como **cifrado**.

Si M es el mensaje que queremos proteger o **texto en claro**, cifrarlo consiste en aplicarle un **algoritmo de cifrado** f , que lo transforma en otro mensaje que llamaremos **texto cifrado**, C . Esto lo podemos expresar como:

$$C = f(M)$$

Para que este cifrado sea útil, debe existir otra transformación o **algoritmo de descifrado** f^{-1} , que permita recuperar el mensaje original a partir del texto cifrado:

$$M = f^{-1}(C)$$

El cifrado de César

Por ejemplo, el “cifrado de César” que hemos mencionado anteriormente consistía en sustituir cada letra del mensaje por la que hay tres posiciones más adelante en el alfabeto (volviendo a empezar por la letra A si llegamos a la Z). De este modo, si aplicamos este algoritmo de cifrado al texto en claro “ALEA JACTA EST” (y utilizando el alfabeto latino actual, porque en tiempos del César no existían letras como la “W”), obtenemos el texto cifrado “DOHD MDFWD HVW”. El descifrado en este caso es muy simple: sólo es necesario sustituir cada letra por la que hay tres posiciones antes en el alfabeto.

Un esquema como el del cifrado de César tiene el inconveniente de que, si el enemigo descubre cuál es el algoritmo de cifrado (y a partir de aquí deduce el algoritmo inver-

Uso de cifrado

El hecho de usar técnicas de cifrado parte de la idea que es muy costoso intentar evitar que un intruso intercepte la información. Enviar mensajes cifrados es más fácil, ya que un espía que consiga verlos no podrá interpretar la información que contienen.

so), será capaz de interpretar todos los mensajes cifrados que capture. Entonces se requeriría instruir a todos los “oficiales de comunicaciones” del ejército para que aprendieran un nuevo algoritmo, lo cual podría resultar complejo. En lugar de ello, lo que se hace en la actualidad es utilizar como algoritmo una función con un parámetro llamado **clave**. 

En este caso podemos hablar de una función de cifrado e con una **clave de cifrado** k , y una función de descifrado d con una **clave de descifrado** x :

$$C = e(k, M)$$

$$M = d(x, C) = d(x, e(k, M))$$

De este modo, una solución al problema del espía que llega a conocer cómo descifrar los mensajes podría ser seguir utilizando el mismo algoritmo, pero con una clave distinta.

Una premisa fundamental en la criptografía moderna es la **suposición de Kerckhoffs**, que establece que los algoritmos deben ser conocidos públicamente y su seguridad solo depende de la clave. En lugar de intentar ocultar el funcionamiento de los algoritmos, es mucho más seguro y efectivo mantener en secreto solamente las claves.

Un algoritmo se considera **seguro** si a un adversario le es imposible obtener el texto en claro M aun conociendo el algoritmo e y el texto cifrado C . Es decir, es imposible descifrar el mensaje sin saber cuál es la clave de descifrado. La palabra “imposible”, debe tomarse en consideración con distintos matices. Un algoritmo criptográfico es **computacionalmente seguro** si, aplicando el mejor método conocido, la cantidad de recursos necesarios (tiempo de cálculo, número de procesadores, etc.) para descifrar el mensaje sin conocer la clave es mucho más grande (unos cuantos órdenes de magnitud) de lo que está al alcance de cualquier persona. En el límite, un algoritmo es **incondicionalmente seguro** si no se puede invertir ni con recursos infinitos. Los algoritmos que se utilizan en la práctica son (o intentan ser) computacionalmente seguros.

La acción de intentar descifrar mensajes sin conocer la clave de descifrado se conoce como “ataque”. Si el ataque tiene éxito, se suele decir coloquialmente que se ha conseguido “romper” el algoritmo. Existen dos formas de llevar a cabo un ataque:

- Mediante el **criptoanálisis**, es decir, estudiando matemáticamente la forma de deducir el texto en claro a partir del texto cifrado.
- Aplicando la **fuerza bruta**, es decir, probando uno a uno todos los valores posibles de la clave de descifrado x hasta encontrar uno que produzca un texto en claro con sentido.

Ejemplo de uso de una clave

El algoritmo de Julio César se puede generalizar definiendo una clave k que indique cuántas posiciones hay que avanzar cada letra en el alfabeto. El cifrado de César utiliza $k = 3$. Para descifrar se puede utilizar el mismo algoritmo pero con la clave invertida ($d \equiv e, x = -k$).

Seguridad por ocultismo

A lo largo de la historia ha habido casos que han demostrado la peligrosidad de basar la protección en mantener los algoritmos en secreto (lo que se conoce como “seguridad por ocultismo”). Si el algoritmo es conocido por muchos, es más fácil que se detecten debilidades o vulnerabilidades y se puedan corregir rápidamente. Si no, un experto podría deducir el algoritmo por ingeniería inversa, y terminar descubriendo que tiene puntos débiles por donde se puede atacar, como sucedió con el algoritmo A5/1 de la telefonía móvil GSM.

Ataques al cifrado de César

Continuando con el ejemplo del algoritmo del César generalizado (con clave), un ataque criptoanalítico podría consistir en el análisis de las propiedades estadísticas del texto cifrado. Las letras cifradas que más se repiten probablemente corresponderán a vocales o a las consonantes más frecuentes, las combinaciones más repetidas de dos (o tres) letras seguidas probablemente corresponden a los dígrafos (o trígrafos) que normalmente aparecen más veces en un texto.

Por otro lado, el ataque por fuerza bruta consistiría en intentar el descifrado con cada uno de los 25 posibles valores de la clave ($1 \leq x \leq 25$, si el alfabeto tiene 26 letras) y mirar cuál de ellos da un resultado inteligible. En este caso, la cantidad necesaria de recursos es tan modesta que incluso se puede realizar el ataque a mano. Por lo tanto, este cifrado sería un ejemplo de algoritmo inseguro o débil.

A continuación veremos las características de los principales sistemas criptográficos utilizados en la protección de las comunicaciones. A partir de ahora, consideraremos los mensajes en claro, los mensajes cifrados y las claves como secuencias de bits.

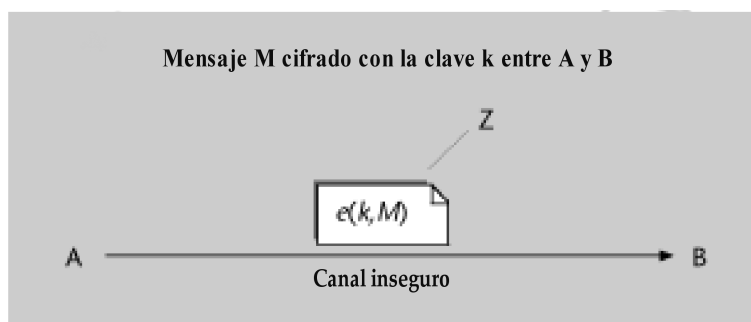
1.1. Criptografía de clave simétrica

Los sistemas criptográficos **de clave simétrica** se caracterizan porque la clave de descifrado x es idéntica a la clave de cifrado k , o bien se puede deducir directamente a partir de ésta.

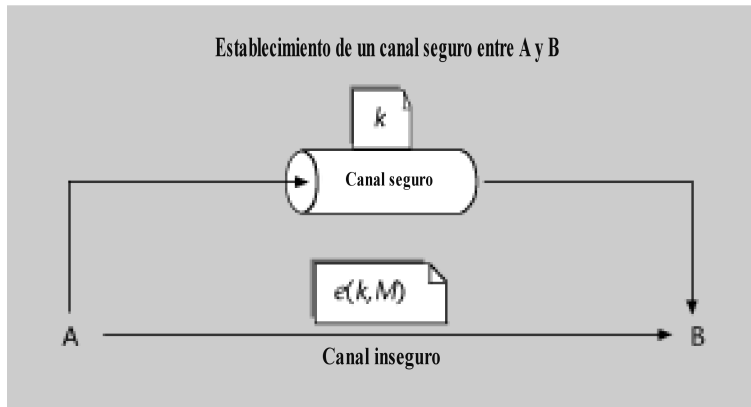
Para simplificar, supondremos que en este tipo de criptosistemas la clave de descifrado es igual a la de cifrado: $x = k$ (si no, siempre podemos considerar que en el algoritmo de descifrado el primer paso es calcular la clave x a partir de k). Es por esto que estas técnicas criptográficas se denominan de *clave simétrica*, o a veces también de **clave compartida**. Así, tenemos:

$$\begin{aligned} C &= e(k, M) \\ M &= d(k, C) = d(k, e(k, M)) \end{aligned}$$

La seguridad del sistema recae pues en mantener en secreto la clave k . Cuando los participantes en una comunicación quieren intercambiarse mensajes confidenciales, tienen que escoger una clave secreta y usarla para cifrar los mensajes. Entonces, pueden enviar estos mensajes por cualquier canal de comunicación, con la confianza que, aunque el canal sea inseguro y susceptible de ser inspeccionado por terceros, ningún espía Z será capaz de interpretarlos.



Si el sistema es de clave compartida, es necesario que el valor de la clave secreta k que usan A y B sea el mismo. ¿Cómo podemos asegurar que esto sea así? Está claro que no pueden mandar la clave escogida mediante el canal de comunicación de que disponen, porque la hipótesis inicial es que este canal es inseguro y todo el mundo podría descubrir la información que se transmite a través del mismo. Una posible solución consiste en utilizar un canal aparte, que se pueda considerar suficientemente seguro:



Esta solución, sin embargo, presenta algunos inconvenientes. Por un lado se supone que el canal seguro no será de uso tan ágil como el canal inseguro (si lo fuera, sería mucho mejor enviar todos los mensajes confidenciales sin cifrar por el canal seguro y olvidarnos del canal inseguro). Por tanto, puede ser difícil ir cambiando la clave. Y por otro lado, este esquema no es suficientemente general: puede ser que tengamos que enviar información cifrada a alguien con quien no podemos contactar de ningún otro modo. Como veremos más adelante, estos problemas relacionados con el **intercambio de claves** se solucionan con la criptografía de clave pública.

A continuación repasaremos las características básicas de los principales algoritmos criptográficos de clave simétrica, que agruparemos en dos categorías: algoritmos de cifrado en flujo y algoritmos de cifrado en bloque.

1.1.1. Algoritmos de cifrado en flujo

El funcionamiento de un cifrado en flujo consiste en la combinación de un texto en claro M con un texto de cifrado S que se obtiene a partir de la clave simétrica k . Para descifrar, sólo se requiere realizar la operación inversa con el texto cifrado y el mismo texto de cifrado S .

La operación de combinación que se utiliza normalmente es la suma, y la operación inversa por tanto es la resta. Si el texto está formado por caracteres, este algoritmo sería como una cifra de César en que la clave va cambiando de un carácter a otro. La clave que corresponde cada vez viene dada por el texto de cifrado S (llamado *keystream* en inglés).

Canales seguros

Podrían ser ejemplos de “canales seguros” el correo tradicional (no electrónico) o un servicio de mensajería “física”, una conversación telefónica, o cara a cara, etc.

Suma y resta de bits

Cuando trabajamos con aritmética binaria o aritmética modulo 2, se cumple que:

$$0 + 0 = 0 \quad 0 - 0 = 0$$

$$0 + 1 = 1 \quad 0 - 1 = 1$$

$$1 + 0 = 1 \quad 1 - 0 = 1$$

$$1 + 1 = 0 \quad 1 - 1 = 0$$

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

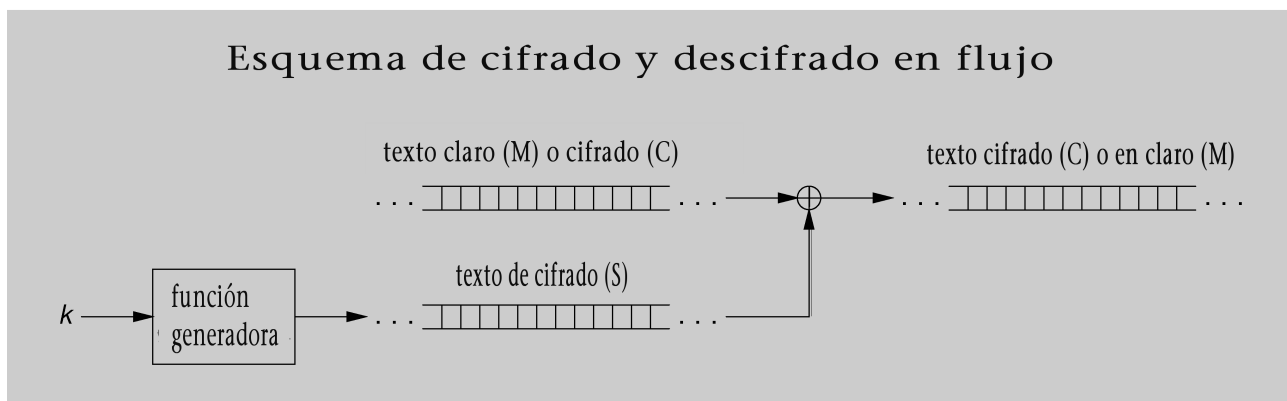
$$1 \oplus 1 = 0$$

Si consideramos el texto formado por bits, la suma y la resta son equivalentes. En efecto, cuando se aplican bit a bit, ambas son idénticas a la operación lógica “O exclusiva”, denotada con el operador XOR (*eXclusive OR*) o el símbolo $\oplus$. Así pues:

$$C = M \oplus S(k)$$

$$M = C \oplus S(k)$$

En los esquemas de cifrado en flujo, el texto en claro M puede ser de cualquier longitud, y el texto de cifrado S ha de ser como mínimo igual de largo. De hecho, no es necesario disponer del mensaje entero antes de empezar a cifrarlo o descifrarlo, ya que se puede implementar el algoritmo para que trabaje con un “flujo de datos” que se va generando a partir de la clave (el texto de cifrado). De ahí procede el nombre de este tipo de algoritmos. La siguiente figura ilustra el mecanismo básico de su implementación:



Existen distintas formas de obtener el texto de cifrado S en función de la clave k :

- Si se escoge una secuencia k más corta que el mensaje M , una posibilidad sería repetirla cíclicamente tantas veces como sea necesario para ir sumándola al texto en claro.

El inconveniente de este método es que se puede romper fácilmente, sobre todo cuanto más corta sea la clave (en el caso mínimo, el algoritmo sería equivalente al cifrado de César).

- En el otro extremo, se podría tomar directamente $S(k) = k$. Esto quiere decir que la propia clave debe ser tan larga como el mensaje que hay que cifrar. Este es el principio del conocido **cifrado de Vernam**. Si k es una secuencia totalmente aleatoria que no se repite cíclicamente, estamos ante de un ejemplo de cifrado incondicionalmente seguro, tal como lo hemos definido al inicio de este módulo. Este método de cifrado se llama en inglés *one-time pad* (“cuaderno de un solo uso”).

El problema es en este caso que el receptor tiene que disponer de la misma secuencia aleatoria para poder realizar el descifrado, y si le tiene que llegar a través de un canal seguro, la pregunta es inmediata: ¿por qué no enviar el mensaje confidencial M , que

Uso del cifrado de Vernam

En ocasiones las comunicaciones entre portaaviones y los aviones utilizan el cifrado de Vernam. En este caso, se aprovecha que en un momento dado (antes del despegue) tanto el avión como el portaaviones están en el mismo sitio, con lo cual, intercambiarse, por ejemplo, un disco duro de 20 GB con una secuencia aleatoria no es ningún problema. Posteriormente, cuando el avión despegue puede establecer una comunicación segura con el portaaviones utilizando un cifrado de Vernam con la clave aleatoria que ambas partes comparten.

es igual de largo que la clave k , directamente por el mismo canal seguro? Es evidente, pues, que este algoritmo es muy seguro pero no es demasiado práctico.

- Lo que en la práctica se utiliza son funciones que generan **secuencias pseudoaleatorias** a partir de una **semilla** (un número que actúa como parámetro del generador), y lo que se intercambia como clave secreta k es solamente esta semilla.

Las secuencias pseudoaleatorias reciben este nombre porque intentan parecer aleatorias pero, obviamente, son generadas algorítmicamente. En cada paso el algoritmo se encontrará en un determinado estado que vendrá dado por sus variables internas. Dado que las variables serán finitas, habrá un número máximo de posibles estados distintos. Esto significa que al cabo de un cierto período, los datos generados se volverán a repetir. Para que el algoritmo sea seguro, interesa que el período de repetición sea cuanto más largo mejor (con relación al mensaje que hay que cifrar), con el fin de dificultar el criptoanálisis. Las secuencias pseudoaleatorias también deben tener otras propiedades estadísticas equivalentes a las de las secuencias aleatorias puras.

Funciones pseudoaleatorias

Son ejemplos de funciones pseudoaleatorias las basadas en registros de desplazamiento realimentados (*feedback shift registers* o FSR). El valor inicial del registro es la semilla. Para ir obteniendo cada bit pseudoaleatorio se desplazan todos los bits del registro una posición y se toma el que sale fuera del registro. El bit que queda libre al otro extremo se llena con un valor que es función del resto de bits.

Cifrado síncrono y asíncrono

Si el texto de cifrado S depende exclusivamente de la clave k , se dice que el cifrado es *síncrono*. Este cifrado tiene el problema de que, si por algún error de transmisión, se pierden bits (o llegan repetidos), el receptor se desincronizará y sumará bits del texto S con bits del texto cifrado C que no corresponden, con lo cual el texto descifrado a partir de entonces será incorrecto.

Esto se puede evitar con el cifrado asíncrono (o auto-sincronizante), en el cual el texto S se calcula a partir de la clave k y el mismo texto cifrado C . Es decir, en lugar de realimentarse con sus propios bits de estado, el generador se realimenta con los últimos n bits cifrados transmitidos. De este modo, si se pierden m bits consecutivos en la comunicación, el error afectará como máximo al descifrado de $m + n$ bits del mensaje original.

Ejemplos de algoritmos de cifrado en flujo

Los algoritmos de cifrado en flujo actualmente en uso tienen la propiedad que son poco costosos de implementar. Las implementaciones en hardware son relativamente simples y, por lo tanto, eficientes en su rendimiento (en términos de bits cifrados por segundo). Pero también las implementaciones en software pueden ser muy eficientes.

Las características del cifrado en flujo lo hacen apropiado para entornos en los que se necesite un rendimiento alto y los recursos (capacidad de cálculo, consumo de energía) sean limitados. Para ello se suelen utilizar en comunicaciones móviles: redes locales sin hilos, telefonía móvil, etc.

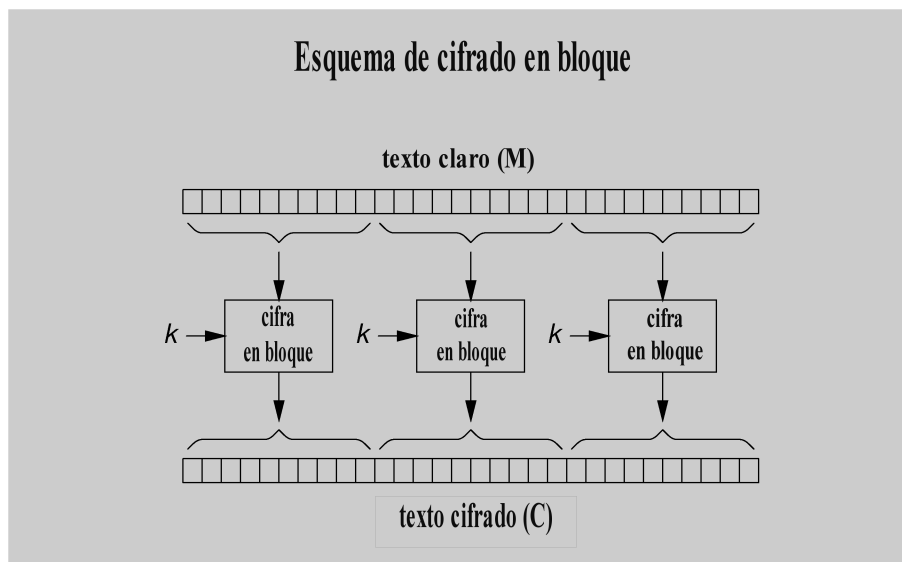
Un ejemplo de algoritmo de cifrado en flujo es el **RC4 (Ron's Code 4)**. Fue diseñado por Ronald Rivest en 1987 y publicado en Internet por un remitente anónimo en 1994. Ha sido el algoritmo de cifrado en flujo más utilizado en muchas aplicaciones gracias a su simplicidad y velocidad. Por ejemplo, el sistema de protección WEP (*Wired Equivalent Privacy*) que originalmente incorporaba el estándar IEEE 802.11 para tecnología LAN inalámbrica utiliza este criptosistema de cifrado en flujo.

1.1.2. Algoritmos de cifrado en bloque

En una cifra de bloque, el algoritmo de cifrado o descifrado se aplica separadamente a bloques de entrada de longitud fija b , y para cada uno de ellos el resultado es un bloque de la misma longitud.

Para cifrar un texto en claro de L bits debemos dividirlo en bloques de b bits cada uno y cifrar estos bloques uno a uno. Si L no es múltiplo de b , se pueden agregar bits adicionales hasta llegar a un número entero de bloques, pero luego puede ser necesario indicar de alguna forma cuántos bits había realmente en el mensaje original. El descifrado también se debe realizar bloque a bloque.

La siguiente figura muestra el esquema básico del cifrado de bloque:



Muchos de los algoritmos del cifrado de bloque se basan en la combinación de dos operaciones básicas: sustitución y transposición.

- La **sustitución** consiste en traducir cada grupo de bits de la entrada a otro, de acuerdo con una permutación determinada.

El cifrado del César sería un ejemplo simple de sustitución, en el que cada grupo de bits correspondería a una letra. De hecho, se trata de un caso particular de **sustitución alfabética**. En el caso más general, las letras del texto cifrado no tienen por qué estar a una distancia constante de las letras del texto en claro (la k del algoritmo, tal como la hemos definido). La clave se puede expresar como la secuencia correlativa de letras que corresponden a la A, la B, la C, etc. Por ejemplo:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Clave: Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Clave de la sustitución alfabética

Está claro que la clave ha de ser una **permutación** del alfabeto, es decir, no puede haber letras repetidas ni faltar ninguna. Si no, la transformación no sería invertible en general.

Texto en claro: A L E A J A C T A E S T

Texto cifrado: Q S T Q P Q E Z Q T L Z

- La **transposición** consiste en reordenar la información del texto en claro según un patrón determinado. Un ejemplo podría ser la formación grupos de cinco letras, incluidos los espacios en blanco, y rescribir cada grupo (1, 2, 3, 4, 5) en el orden (3, 1, 5, 4, 2):

Texto en claro: A L E A _ J A C T A _ E S T _

Texto cifrado: E A _ A L C J A T A S _ _ T E

La transposición por sí sola no dificulta extraordinariamente el criptoanálisis, pero puede combinarse con otras operaciones para añadir complejidad a los algoritmos de cifrado.

El **producto de cifras**, o combinación en cascada de distintas transformaciones criptográficas, es una técnica muy efectiva para implementar algoritmos bastante seguros de forma sencilla. Por ejemplo, muchos algoritmos de cifrado de bloque se basan en una serie de iteraciones de productos sustitución–transposición.

Confusión y difusión

Dos propiedades deseables en un algoritmo criptográfico son la *confusión*, que consiste en esconder la relación entre la clave y las propiedades estadísticas del texto cifrado, y la *difusión*, que propaga la redundancia del texto en claro a lo largo del texto cifrado para que no sea fácilmente reconocible.

La confusión consigue que, cambiando un solo bit de la clave, cambien muchos bits del texto cifrado, y la difusión implica que el cambio de un solo bit del texto en claro afecte también a muchos bits del texto cifrado.

En un bucle de productos de cifrados básicos, la sustitución contribuye a la confusión, mientras que la transposición contribuye a la difusión. La combinación de estas transformaciones simples, repetidas diversas veces, provoca que los cambios en la entrada se propaguen por toda la salida por efecto “avalancha”.

Ejemplos de algoritmos de cifrado en bloque

DES (Data Encryption Standard). Durante muchos años ha sido el algoritmo más estudiado y, a la vez, el más utilizado. Desarrollado por IBM durante los años 70, fue adoptado en 1977 por el NBS norteamericano (nombre que tenía entonces el actual NIST) como estándar para el cifrado de datos.

El algoritmo admite una clave de 64 bits, pero sólo 7 de cada 8 intervienen en el cifrado, de modo que la longitud efectiva de la clave es de 56 bits. Los bloques de texto a los que se les aplica el DES tienen que ser de 64 bits cada uno.

La parte central del algoritmo consiste en dividir la entrada en grupos de bits, hacer una sustitución distinta sobre cada grupo y, a continuación una transposición de todos los bits. Esta transformación se repite dieciséis veces: en cada iteración, la entrada es una transposición distinta de los bits de la clave sumada bit a bit (XOR) con la salida de la iteración anterior. Tal como está diseñado el algoritmo, el descifrado se realiza igual que el cifra-

NBS y NIST

NBS era la sigla de *National Bureau of Standards*, y NIST es la sigla de *National Institute of Standards and Technology*.

Bits adicionales de la clave DES

Un posible uso de los bits de la clave DES que no influyen en el algoritmo es su utilización como bits de paridad.

do pero realizando las transposiciones de la clave en el orden inverso (empezando por la última).

Triple DES. Aunque a lo largo de los años el algoritmo DES se ha mostrado muy resistente al criptoanálisis, su principal problema es actualmente la vulnerabilidad a los ataques de fuerza bruta, a causa de la longitud de la clave, de sólo 56 bits. Aunque en los años 70 era muy costoso realizar una búsqueda entre las 2^{56} combinaciones posibles, la tecnología actual permite romper el algoritmo en un tiempo cada vez más corto. Por este motivo, en 1999 el NIST cambió el algoritmo DES por el “Triple DES” como estándar oficial, mientras no estuviera disponible el nuevo estándar AES. El Triple DES, como su nombre indica, consiste en aplicar el DES tres veces consecutivas. Esto se puede realizar con tres claves (k_1, k_2, k_3) , o bien con sólo dos (k_1, k_2) , y otra vez k_1 . La longitud total de la clave con la segunda opción es de 112 bits (dos claves de 56 bits), que entonces ya se consideraba suficientemente segura; la primera opción proporciona más seguridad, pero a costa de utilizar una clave total de 168 bits (3 claves de 56 bits), que puede ser un poco más difícil de gestionar e intercambiar.

Para conseguir que el sistema sea adaptable al estándar antiguo, en el Triple DES se aplica una secuencia cifrado-descifrado-cifrado (E-D-E) en lugar de tres cifrados:

$$C = e(k_3, d(k_2, e(k_1, M)))$$

$$\text{o bien: } C = e(k_1, d(k_2, e(k_1, M)))$$

Así, tomando $k_2 = k_1$ tenemos un sistema equivalente al DES simple.

AES (Advanced Encryption Standard). Dado que el estándar DES empezaba a quedarse anticuado, a causa sobretudo de la longitud tan corta de sus claves, y el Triple DES no es excesivamente eficiente cuando se implementa con software, en 1997 el NIST convocó a la comunidad criptográfica a presentar propuestas para un nuevo estándar, el AES, que sustituyera al DES. De los quince algoritmos candidatos que se aceptaron, se escogieron cinco como finalistas, y en octubre de 2000 se dio a conocer el ganador: el algoritmo Rijndael, propuesto por los criptógrafos belgas Joan Daemen y Vincent Rijmen.

El Rijndael puede trabajar en bloques de 128, 192 o 256 bits (aunque el estándar AES sólo prevé los de 128), y la longitud de la clave también puede ser de 128, 192 o 256 bits. Dependiendo de esta última longitud, el número de iteraciones del algoritmo es 10, 12 ó 14, respectivamente. Cada iteración incluye una sustitución fija byte a byte, una transposición, una transformación consistente en desplazamientos de bits y XORs, y una suma binaria (XOR) con bits obtenidos a partir de la clave.

1.1.3. Uso de los algoritmos de clave simétrica

Cuando se utiliza el cifrado simétrico para proteger las comunicaciones, se puede escoger el algoritmo que sea más apropiado a las necesidades de cada aplicación: normalmente, a más seguridad menos velocidad de cifrado, y viceversa.

Los retos DES

En la dirección cryptography.fandom.com/wiki/DES_Challenges se puede encontrar información sobre los “retos DES”, que demuestra que en 1999 ya era posible romper una clave DES en menos de 24 horas.

Repetición del DES

La operación de aplicar el cifrado DES con una clave, y el resultado volverlo a cifrar con otra clave, no es equivalente a un solo cifrado DES (no hay ninguna clave única que dé el mismo resultado que dan las otras dos juntas). Si no fuera de este modo, la repetición del DES no sería más segura que el DES simple.

Doble DES

El motivo por el que el algoritmo DES se aplica tres veces, y no dos, es porque un atacante que tuviera suficientes recursos podría romper el “Doble DES” por fuerza bruta, mediante un ataque de texto en claro conocido, con menos de 2^{57} operaciones de cifrado en vez de las 2^{112} que cabría esperar.

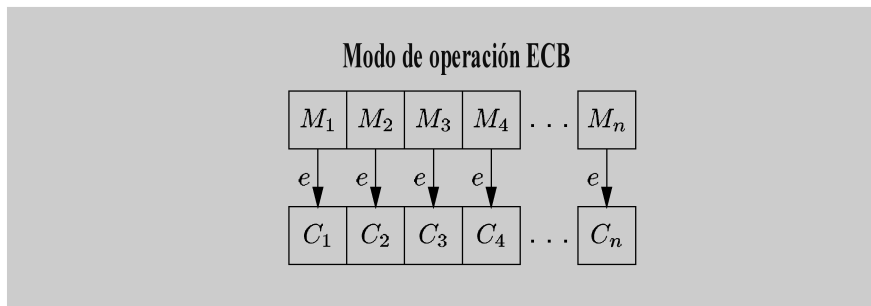
Un aspecto que hay que tener en cuenta es que, aunque el cifrado puede conseguir que un atacante no descubra directamente los datos transmitidos, en ocasiones es posible que se pueda deducir información indirectamente. Por ejemplo, en un protocolo que utilice mensajes con una cabecera fija, la aparición de los mismos datos cifrados varias veces en una transmisión puede indicar dónde empiezan los mensajes.

Esto no pasa con el cifrado en flujo si su periodo es lo suficientemente largo, pero en un cifrado en bloque, si dos bloques de texto en claro son iguales y se utiliza la misma clave, los bloques cifrados también serán iguales. Para contrarrestar esta propiedad, se pueden aplicar distintos **modos de operación** al cifrado en bloque.

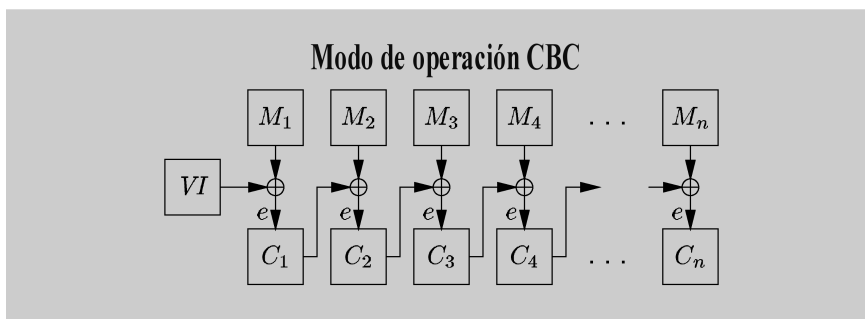
- El modo ECB (*Electronic Codebook*) es el más simple, y consiste en dividir el texto en bloques y cifrar cada uno de ellos de forma independiente. Este modo adolece del problema de dar bloques iguales cuando en la entrada existen bloques iguales.

Modo ECB

El nombre del modo ECB (*Electronic Codebook*) da la idea que se puede considerar como una simple sustitución bloque a bloque, de acuerdo con un código o diccionario (con muchas entradas, sin duda) que viene dado por la clave.



- En el modo CBC (*Cipher Block Chaining*), se suma a cada bloque de texto en claro, antes de cifrarlo, (bit a bit, con XOR) el bloque cifrado anterior. Al primer bloque se le suma un **vector de inicialización** (VI), que es un conjunto de bits aleatorios de la misma longitud que un bloque. Escogiendo vectores distintos cada vez, aunque el texto en claro sea el mismo, los datos cifrados serán distintos. El receptor debe conocer el valor del vector antes de empezar a descifrar, pero no es necesario guardar este valor en secreto, sino que normalmente se transmite como cabecera del texto cifrado.

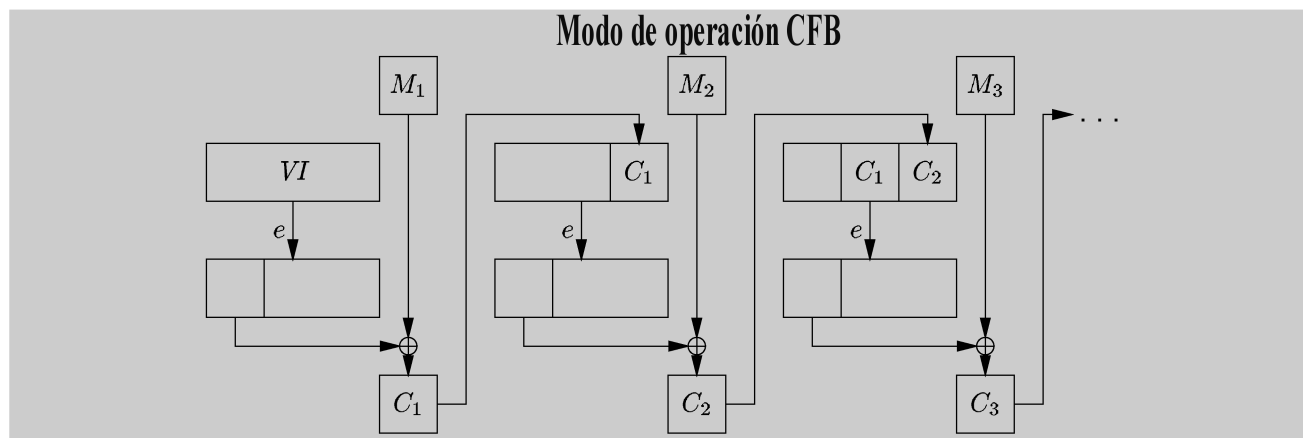


Modo CFB como cifrado en flujo

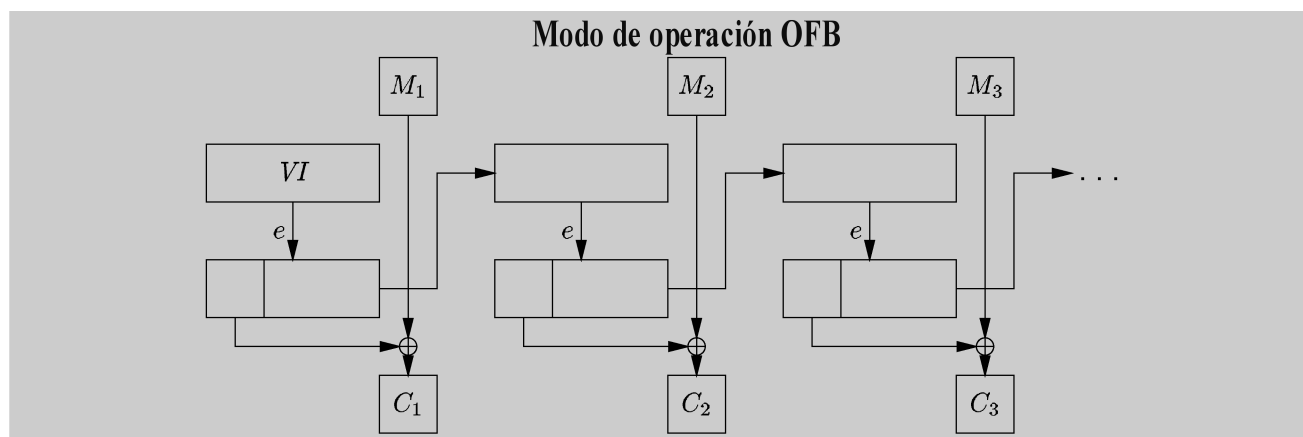
- En el modo CFB (*Cipher Feedback*), el algoritmo de cifrado no se aplica directamente al texto en claro sino a un vector auxiliar (inicialmente igual al VI). Del resultado del cifrado se toman n bits que se suman a n bits del texto en claro para obtener n bits de texto cifrado. Estos bits cifrados se utilizan también para actualizar el vector auxiliar. El número n de bits generados en cada iteración puede ser menor o igual que la longitud

Es fácil ver que el modo CFB (y también el OFB) se puede considerar como un cifrado en flujo que utiliza como función generadora un cifrado en bloque.

de bloque b . Tomando como ejemplo $n = 8$, tenemos un cifrado que genera un byte cada vez sin que sea necesario esperar a tener un bloque entero para poderlo cifrar.



- El modo OFB (*Output Feedback*) opera como el CFB pero en lugar de actualizar el vector auxiliar con el texto cifrado, se actualiza con el resultado obtenido del algoritmo de cifrado. La propiedad que distingue este modo de los demás consiste en que un error en la recuperación de un bit cifrado afecta solamente al descifrado de este bit.



- A partir de los modos anteriores se pueden definir varias variantes. Por ejemplo, el modo CTR (*Counter*) es como el OFB, pero el vector auxiliar no se realimenta con el cifrado anterior sino que simplemente es un contador que se va incrementando.

Existe otra técnica para evitar que textos de entrada iguales produzcan textos cifrados iguales, que se puede aplicar también a cifrados que no utilizan vector de inicialización (incluido el cifrado en flujo). Esta técnica consiste en la modificación de la clave secreta con bits aleatorios antes de usarla en el algoritmo de cifrado (o en el de descifrado). Como estos bits aleatorios sirven para dar un “sabor” distinto a la clave, se les suele llamar **bits de sal**. Igual que el vector de inicialización, los bits de sal se envían en claro antes que el texto cifrado.

1.1.4. Funciones *hash* seguras

Aparte de cifrar datos, existen algoritmos basados en técnicas criptográficas que se usan para garantizar la autenticidad de los mensajes. Un tipo de algoritmos de estas características son las llamadas **funciones *hash* seguras**, también conocidas como funciones de **resumen de mensaje** (*message digest*, en inglés).

En general, podemos decir que una función *hash* nos permite obtener una cadena de bits de longitud fija, relativamente corta, a partir de un mensaje de longitud arbitraria:

$$H = h(M)$$

Para mensajes M iguales, la función h debe dar resúmenes H iguales. Pero si dos mensajes dan el mismo resumen H no deben ser necesariamente iguales. Esto es así porque sólo existe un conjunto limitado de posibles valores H , ya que su longitud es fija, y en cambio puede haber muchos más mensajes M (si la longitud puede ser cualquiera, habrá infinitos).

Para poderla aplicar en un sistema de autenticación, la función h debe ser una función *hash* segura. 

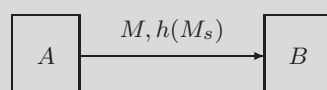
Se entiende que una función *hash* o de resumen es **segura** si cumple las siguientes condiciones:

- Es **unidireccional**, es decir, si tenemos $H = h(M)$ es computacionalmente inviable encontrar M a partir del resumen H .
- Es **resistente a colisiones**, es decir, dado un mensaje M cualquiera es computacionalmente inviable encontrar un mensaje $M' \neq M$ tal que $h(M') = h(M)$.

Secreto de los algoritmos

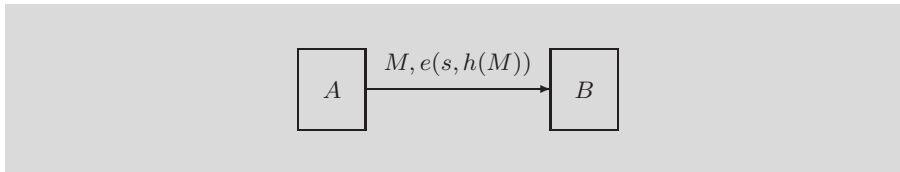
Observad que las funciones *hash* son conocidas, puesto que todo el mundo debe poder calcular los resúmenes del mismo modo.

Estas propiedades permiten el uso de las funciones *hash* seguras para dar un servicio de autenticidad basado en una clave secreta s compartida entre dos partes A y B . Aprovechando la unidireccionalidad, cuando A quiere mandar un mensaje M a B puede preparar otro mensaje M_s : por ejemplo, concatenando el original con la clave: $M_s = (M, s)$. Entonces manda a B el mensaje M y el resumen del mensaje M_s :



Para comprobar la autenticidad del mensaje recibido, B verifica que el resumen corresponda efectivamente a M_s . Si es así, quiere decir que lo ha generado alguien que conoce la clave secreta s (que debería ser A), y también que nadie ha modificado el mensaje.

Otra técnica consistiría en calcular el resumen del mensaje M y cifrarlo utilizando s como clave de cifrado:



Para verificar la autenticidad se debe recuperar el resumen enviado, descifrándolo con la clave secreta s , y compararlo con el resumen del mensaje M . Un atacante que quisiera modificar el mensaje sin conocer la clave podría intentar sustituirlo por otro que diera el mismo resumen, con lo cual B no detectaría la falsificación. Pero si la función de resumen es resistente a colisiones, esto le sería imposible al atacante.

Para dificultar los ataques contra las funciones de resumen, por un lado los algoritmos tienen que definir una relación compleja entre los bits de entrada y cada bit de salida. Por otro lado, los ataques por fuerza bruta se contrarrestan alargando lo suficiente la longitud del resumen. Por ejemplo, los algoritmos usados actualmente generan resúmenes de 128 ó 160 bits. Esto quiere decir que un atacante podría tener que probar del orden de 2^{128} o 2^{160} mensajes de entrada para encontrar una colisión (es decir, un mensaje distinto que diera el mismo resumen).

Pero existe otro tipo de ataque más ventajoso para el atacante, llamado **ataque del cumpleaños**. Un ataque de este tipo parte de la suposición de que el atacante puede escoger el mensaje que será autenticado. La víctima lee el mensaje y, si lo acepta, lo autentica con su clave secreta. Pero el atacante ha presentado este mensaje porque ha encontrado otro que da el mismo resumen y, por lo tanto, puede hacer creer al destinatario que el mensaje auténtico es este otro. Y esto se puede conseguir realizando una búsqueda por fuerza bruta con muchas menos operaciones: del orden de 2^{64} ó 2^{80} , si el resumen es de 128 ó 160 bits, respectivamente.

Paradoja del cumpleaños

El nombre de este tipo de ataque viene de un problema clásico de probabilidades conocido como la “paradoja del cumpleaños”. El problema consiste en encontrar el número mínimo de personas que debe haber en un grupo para que la probabilidad de que como mínimo dos de ellas celebren su aniversario el mismo día sea superior al 50 %. Una respuesta intuitiva puede ser que la solución sea del orden de 200, pero este resultado no es correcto. Podría ser correcto si se quisiera obtener el número de personas necesarias para obtener un 50 % de probabilidad de coincidencia con *una* persona determinada. Si permitimos que la coincidencia sea entre *cualquier* pareja de personas, la solución es un número mucho menor: 23.

La conclusión es que si una función de resumen puede dar N valores distintos, para que la probabilidad de encontrar dos mensajes con el mismo resumen sea del 50 % el número de mensajes que hay que probar es del orden de $\sqrt{N}$.

Autenticidad y confidencialidad

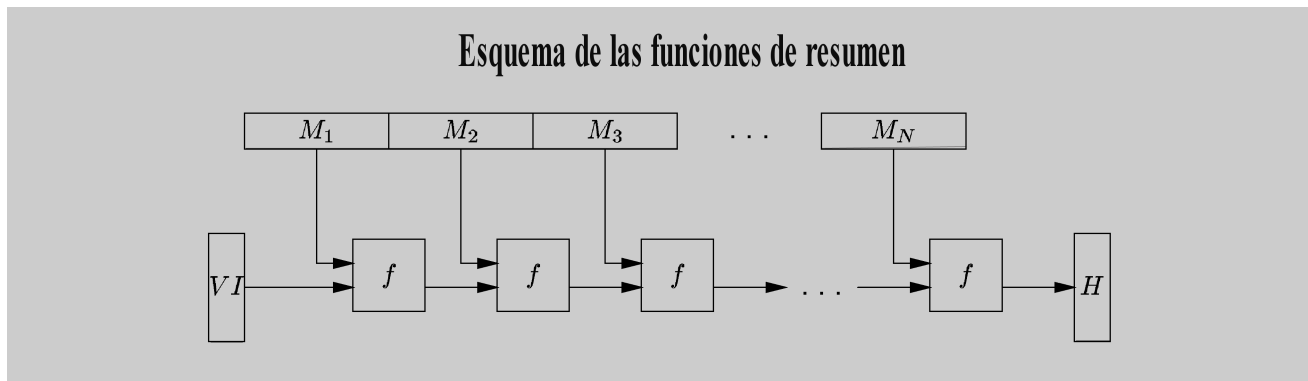
Cifrar solamente el resumen, en lugar del mensaje entero, es más eficiente porque hay que cifrar menos bits. Esto, evidentemente, suponiendo que solamente se precise autenticidad, y no confidencialidad. Si también interesa que el mensaje sea confidencial, entonces sí es necesario cifrarlo entero.

Resistencia fuerte a las colisiones

La resistencia de los algoritmos de resumen a las colisiones, tal como la hemos definido, a veces recibe el nombre de “resistencia débil”, mientras que la propiedad de ser resistente a ataques de cumpleaños se conoce como “resistencia fuerte”.

Ejemplos de funciones *hash* seguras

El esquema de la mayoría de funciones *hash* usadas actualmente es parecido al de los algoritmos de cifrado de bloque: el mensaje de entrada se divide en bloques de la misma longitud, y a cada uno se le aplica una serie de operaciones junto con el resultado obtenido en el bloque anterior. El resultado que queda después de procesar el último bloque es el resumen del mensaje.



El objetivo de estos algoritmos es que cada bit de salida dependa de todos los bits de entrada. Esto se consigue con diferentes iteraciones de operaciones que “mezclan” los bits entre ellos, de forma parecida a cómo la sucesión de transposiciones en los cifrados de bloque provoca un “efecto avalancha” que garantiza la difusión de los bits.

Hasta hace poco, el algoritmo de *hash* más usado era el **MD5 (Message Digest 5)**. Pero como el resumen que da es de sólo 128 bits, y aparte se han encontrado otras formas de generar colisiones parciales en el algoritmo, actualmente se recomienda utilizar algoritmos más seguros, como el **SHA-1 (Secure Hash Algorithm-1)**. El algoritmo SHA-1, publicado el 1995 en un estándar del NIST (como revisión de un algoritmo anterior llamado simplemente SHA), da resúmenes de 160 bits. El año 2002 el NIST publicó variantes de este algoritmo que generan resúmenes de 256, 384 y 512 bits.

Longitud del resumen MD5

Como la longitud del resumen MD5 es de 128 bits, el número de operaciones para un ataque de aniversario es del orden de 2^{64} . Comparad esta magnitud con la de un ataque por fuerza bruta contra el DES (menos de 2^{56} operaciones), de la que no está demasiado lejos.

1.2. Criptografía de clave pública

1.2.1. Algoritmos de clave pública

Como hemos visto en el subapartado anterior, uno de los problemas de la criptografía de clave simétrica es el de la distribución de las claves. Este problema se puede solucionar si utilizamos **algoritmos de clave pública**, también llamados **de clave asimétrica**.

En un algoritmo criptográfico de clave pública se utilizan claves distintas para el cifrado y el descifrado. Una de ellas, la **clave pública**, se puede obtener fácilmente a partir de la otra, la **clave privada**, pero por el contrario es computacionalmente de muy difícil obtención la clave privada a partir de la clave pública.

Los algoritmos de clave pública típicos permiten cifrar con la clave pública (k_{pub}) y descifrar con la clave privada (k_{pr}):

$$C = e(k_{\text{pub}}, M)$$

$$M = d(k_{\text{pr}}, C)$$

Pero también puede haber algoritmos que permitan cifrar con la clave privada y descifrar con la pública (más adelante veremos cómo se puede utilizar esta propiedad):

$$C = e(k_{\text{pr}}, M)$$

$$M = d(k_{\text{pub}}, C)$$

Los algoritmos de clave pública se basan en problemas matemáticos “fáciles” de plantear a partir de la solución, pero “difíciles” de resolver. En este contexto, se entiende que un problema es fácil si el tiempo para resolverlo, en función de la longitud n de los datos, se puede expresar en forma polinómica como, por ejemplo $n^2 + 2n$ (en teoría de la complejidad, se dice que estos problemas son de la “clase P”). Si el tiempo de resolución crece más rápidamente, como por ejemplo con 2^n , el problema se considera difícil. Así, se puede escoger un valor de n tal que el planteamiento sea viable pero la resolución sea computacionalmente intratable.

Un ejemplo de problema fácil de plantear pero difícil de resolver es el de los logaritmos discretos. Si trabajamos con aritmética módulo m , resulta fácil calcular esta expresión:

$$y = b^x \bmod m$$

El valor x se llama logaritmo discreto de y en base b módulo m . Escogiendo convenientemente b y m , puede ser difícil calcular el logaritmo discreto de cualquier y . Una posibilidad es ir probando todos los valores de x : si m es un número de n bits, el tiempo para encontrar la solución aumenta proporcionalmente a 2^n . Hay otros métodos más eficientes para calcular logaritmos discretos, pero el mejor algoritmo conocido también necesita más tiempo del que se puede expresar polinómicamente.

Adaptación de los problemas difíciles

Si los adelantos de la tecnología reducen el tiempo de resolución, se puede aumentar la longitud n , con lo cual se necesitarán unas cuantas operaciones más para el planteamiento, pero la complejidad de la solución crecerá exponencialmente.

Ejemplo de operaciones módulo m

Por ejemplo, para obtener $14^{11} \pmod{19}$ podemos multiplicar 11 veces el número 14, dividir el resultado entre 19 y tomar el residuo de la división, que es igual a 13. Pero también se puede aprovechar que el exponente 11 es 1011 en binario ($11 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$), y entonces $14^{11} = 14^8 \cdot 14^2 \cdot 14^1$, para obtener el resultado con menos multiplicaciones:

$$\begin{aligned} 14^1 &= 14 && \equiv 14 \pmod{19} \rightarrow 14 \\ 14^2 &= 14^1 \times 14^1 \equiv 14 \times 14 = 196 \equiv 6 \pmod{19} \rightarrow 6 \\ 14^4 &= 14^2 \times 14^2 \equiv 6 \times 6 = 36 \equiv 17 \pmod{19} \\ 14^8 &= 14^4 \times 14^4 \equiv 17 \times 17 = 289 \equiv 4 \pmod{19} \rightarrow 4 \\ &&& 336 \equiv 13 \pmod{19} \end{aligned}$$

Así, sabemos que $\log_{14} 13 = 11 \pmod{19}$. Pero si tuviéramos que calcular el logaritmo de cualquier otro número y tendríamos que ir probando uno a uno los exponentes hasta encontrar uno que dé como resultado y . Y si en lugar de tratarse de números de 4 o 5 bits como estos fueran números de más de 1000 bits, el problema sería intratable.

Así pues, los algoritmos de clave pública han de ser diseñados de manera que sea inviable calcular la clave privada a partir de la pública, y lógicamente también ha de ser inviable invertirlos sin saber la clave privada, pero el cifrado y el descifrado se han de poder realizar en un tiempo relativamente corto.

A la práctica, los algoritmos utilizados permiten cifrar y descifrar fácilmente, pero todos ellos son considerablemente más lentos que los equivalentes con criptografía simétrica. Por eso, la criptografía de clave pública se suele utilizar solo en los problemas que la criptografía simétrica no puede resolver: el intercambio de claves y la autenticación con no repudio (firmas digitales). 

Velocidad de la criptografía de clave pública

El cifrado y descifrado con algoritmos de clave pública puede llegar a ser dos o tres órdenes de magnitud más lentos que con criptografía simétrica.

- Los mecanismos de **intercambio de claves** permiten que dos partes se pongan de acuerdo en las claves simétricas que utilizarán para comunicarse, sin que un tercero que esté escuchando el diálogo pueda deducir cuáles son estas claves.

Por ejemplo, A puede escoger una clave simétrica k , cifrarla con la clave pública de B , y enviar el resultado a B . Luego B descifrará con su clave privada el valor recibido, y sabrá cuál es la clave k que ha escogido A . El resto de la comunicación irá cifrada con un algoritmo simétrico (mucho más rápido), y utilizará esta clave k . Los atacantes, al no conocer la clave privada de B , no podrán deducir el valor de k .

- La **autenticación** basada en clave pública se puede utilizar si el algoritmo permite utilizar las claves a la inversa: la clave privada para cifrar y la clave pública para descifrar. Si A envía un mensaje cifrado con su clave privada, todo el mundo podrá descifrarlo con la clave pública de A , y al mismo tiempo todo el mundo sabrá que el mensaje sólo lo puede haber generado quien conozca la clave privada asociada (que debería ser A). Ésta es la base de las **firmas digitales**.

Ejemplos de algoritmos de clave pública

Intercambio de claves Diffie-Hellman. Es un mecanismo que permite que dos partes se pongan de acuerdo de forma segura sobre una clave secreta. El algoritmo se basa en la dificultad de calcular logaritmos discretos. Estos mecanismos fueron propuestos por Whit-

field Diffie y Martin Hellman el año 1976. Dos participantes escogen sus claves privadas a y b y hacen públicos los valores α^a y α^b en aritmética módulo n . Los dos pueden calcular $\alpha^{a \cdot b}$ exponenciando a su clave privada la clave pública del otro, y pueden utilizar este valor como clave secreta entre ellos. Dada la dificultad de calcular logaritmos discretos, nadie más puede deducir cual es este valor.

RSA. Es el algoritmo más utilizado en la historia de la criptografía de clave pública. Su nombre procede de las iniciales de quienes lo diseñaron en 1977: Ronald Rivest, Adi Shamir y Leonard Adleman. La clave pública está formada por un número n , calculado como producto de dos factores primos muy grandes ($n = p \cdot q$) y un exponente e . La clave privada es otro exponente d calculado a partir de p , q y e , de tal forma que el cifrado y el descifrado se puede realizar de la siguiente forma:

$$\begin{aligned}\text{Cifrado: } C &= M^e \text{ mód } n \\ \text{Descifrado: } M &= C^d \text{ mód } n\end{aligned}$$

Como se puede ver, la clave pública y la privada son intercambiables: si se usa cualquiera de ellas para cifrar, se deberá utilizar la otra para descifrar.

La fortaleza del algoritmo RSA se basa, por un lado, en la dificultad de obtener M a partir de C sin conocer d (problema del logaritmo discreto), y por otro lado, en la dificultad de obtener p y q (y, por tanto, d) a partir de n (problema de la factorización de números grandes, que es otro de los problemas considerados difíciles).

ElGamal. Otro esquema de cifrado, en este caso, basado en el problema de Diffie-Hellman.

DSA (Digital Signature Algorithm). Publicado por el NIST en distintas versiones del *Digital Signature Standard* (DSS), la primera de ellas en 1991. Es una variante del algoritmo de firma ElGamal, que a su vez sigue el esquema de cifrado de ElGamal. El DSA no es un algoritmo de cifrado, sino que sólo permite generar firmas y verificarlas.

Curvas elípticas. Mientras que los algoritmos anteriores trabajan básicamente con productos de números muy grandes en aritmética modular, esta nueva rama de la criptografía de clave pública utiliza operaciones definidas sobre puntos de curvas elípticas en un espacio de coordenadas enteras bidimensionales. La ventaja de esta técnica es que proporciona una seguridad equivalente a la de los otros algoritmos con muchos menos bits de clave, y con un tiempo de cálculo también mucho menor.

1.2.2. Uso de la criptografía de clave pública

Hemos visto antes que las principales aplicaciones de la criptografía de clave pública son el intercambio de claves para proporcionar confidencialidad y la firma digital para proporcionar autenticidad y no repudio.

Valores usados en el RSA

Actualmente el problema de factorizar números de 512 bits es muy complejo, aunque abordable si se dispone de suficientes recursos. Por lo tanto, se recomienda utilizar claves públicas con un valor n a partir de 1.024 bits. Como exponente público e normalmente se utilizan valores simples como 3 ó 65.537 ($2^{16} + 1$) porque hacen más rápido el cifrado.

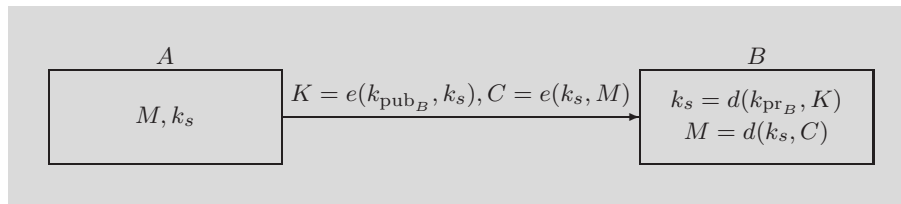
Seguridad con curvas elípticas

La seguridad que proporciona una clave RSA de 1024 bits se puede conseguir con menos de 170 bits de clave cuando se utiliza criptografía de curvas elípticas.

- El problema de la confidencialidad entre dos partes que sólo disponen de un canal inseguro para comunicarse se resuelve con la criptografía de clave pública. Cuando A quiere mandar un mensaje secreto M a B , no hace falta cifrar todo el mensaje con un algoritmo de clave pública (esto podría resultar muy lento), sino que se escoge una clave simétrica k_s , en ocasiones llamada **clave de sesión** o **clave de transporte**, y se cifra el mensaje con un algoritmo simétrico usando esta clave. Lo único que hace falta cifrar con la clave pública de B (k_{pub_B}) es la clave de sesión. En recepción, B utiliza su clave privada (k_{pr_B}) para recuperar la clave de sesión k_s , y entonces ya puede descifrar el mensaje cifrado.

Sobre digital

Como veremos más adelante, esta técnica para proporcionar confidencialidad con criptografía de clave pública se suele llamar “sobre digital” en el contexto del correo electrónico seguro.

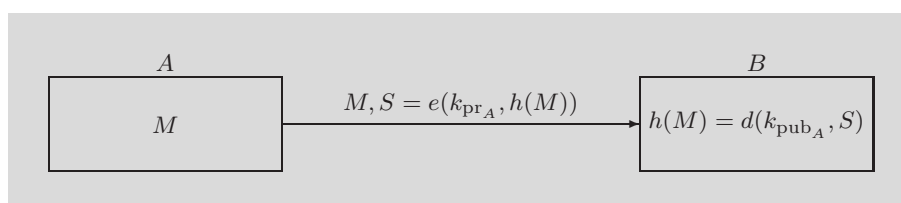


Dado que la clave de sesión es un mensaje relativamente corto (por ejemplo, si es una clave DES sólo tendrá 56 bits), un atacante podría intentar romper el cifrado por fuerza bruta, pero no intentando descifrar el mensaje con los posibles valores de la clave privada k_{pr_B} , sino cifrando los posibles valores de la clave de sesión k_s con la clave pública k_{pub_B} . En el caso de una clave de sesión DES, independientemente del número de bits de la clave pública, el atacante solamente necesitaría un esfuerzo del orden de 2^{56} operaciones.

Para evitar este tipo de ataque, la información que se cifra realmente con la clave pública no es directamente el valor secreto (en este caso k_s), sino que a este valor se le añade una cadena más o menos larga de bits aleatorios. El receptor solamente tiene que descartar estos bits aleatorios del resultado que obtenga del descifrado.

- Una **firma digital** es básicamente un mensaje cifrado con la clave privada del firmante. Pero, por cuestiones de eficiencia, lo que se cifra no es directamente el mensaje a firmar, sino solamente su resumen calculado con una función *hash* segura.

Cuando A quiera mandar un mensaje firmado, tendrá que obtener su resumen y cifrarlo con la clave privada k_{pr_A} . Para verificar la firma, el receptor tiene que descifrarla con la clave pública k_{pub_A} y comparar el resultado con el resumen del mensaje: si son iguales, quiere decir que el mensaje lo ha generado A y nadie lo ha modificado. Dado que se supone que la función de resumen es resistente a colisiones, un atacante no podrá modificar el mensaje sin que la firma deje de ser válida.



1.3. Infraestructura de clave pública (PKI)

Como hemos visto hasta ahora, la criptografía de clave pública permite resolver el problema del intercambio de claves, utilizando las claves públicas de los participantes. Pero se plantea otro problema: si alguien afirma ser A y su clave pública es k_{pub} , ¿cómo podemos saber que realmente k_{pub} es la clave pública de A ? Porque es perfectamente posible que un atacante Z genere su par de claves $(k'_{\text{pr}}, k'_{\text{pub}})$ y afirme “yo soy A , y mi clave pública es k'_{pub} ”.

Una posible solución a este problema es que exista una entidad de confianza que nos asegure que, efectivamente, las claves públicas pertenecen a sus supuestos propietarios. Esta entidad puede firmar un documento que afirme “la clave pública de A es k_{pub_A} ”, y publicarlo para que todos los usuarios lo sepan. Este tipo de documento se llama **certificado de clave pública** o **certificado digital**, y es la base de lo que se conoce como **infraestructura de clave pública**.

1.3.1. Certificados de clave pública

Un certificado de clave pública o certificado digital consta de tres partes básicas:

- Una identificación de usuario como, por ejemplo, su nombre.
- El valor de la clave pública de este usuario.
- La firma de las dos partes anteriores.

Si el autor de la firma es alguien en quien confiamos, el certificado nos sirve como garantía de que la clave pública pertenece al usuario que figura identificado en el certificado. Quien firma el certificado puede ser una autoridad que se responsabilice de verificar fehacientemente la autenticidad de las claves públicas. En este caso, se dice que el certificado ha sido generado por una **autoridad de certificación** (CA).

Puede haber distintos formatos de certificados, pero el más usado es el de los **certificados X.509**, especificado en la definición del **servicio de directorio X.500**.

El directorio X.500 permite almacenar y recuperar información, expresada como **atributos**, de un conjunto de **objetos**. Los objetos X.500 pueden representar, por ejemplo, países, ciudades, o bien empresas, universidades (en general, organizaciones), departamentos, facultades (en general, unidades organizativas), personas, etc. Todos estos objetos están organizados jerárquicamente en forma de árbol (en cada nodo del árbol existe un objeto) y, dentro de cada nivel, los objetos se identifican mediante un **atributo distintivo**. A nivel global, cada objeto se identifica con un **nombre distintivo** (DN), que no es más que la concatenación de los atributos distintivos que hay entre la raíz del árbol y el objeto en cuestión. El sistema de nombres es, pues, parecido al DNS de Internet, con la diferencia que los componentes de un nombre DNS son simples cadenas de caracteres, y los de un DN X.500 son atributos, cada uno con un tipo y un valor.

PKI

La sigla PKI corresponde al nombre en inglés de la infraestructura de clave pública (*Public Key Infrastructure*).

CA

CA es la sigla inglesa de *Certification Authority*.

El directorio X.500

La especificación del directorio X.500 está publicada en la Serie de Recomendaciones ITU-T X.500, una de las cuales es la Recomendación X.509.

Algunos ejemplos de tipos de atributos que se pueden usar como atributos distintivos en un DN son: *countryName* (habitualmente denotado con la abreviatura “c”), *stateOrProvinceName* (“st”), *localityName* (“l”), *organizationName* (“o”), *organizationalUnitName* (“ou”), *commonName* (“cn”), *surname* (“sn”), etc. Un ejemplo de DN es “c=ES, st=Barcelona, l=Barcelona, o=Universitat Oberta de Catalunya, ou=SI, cn=cv.uoc.edu”.

DN

DN es la sigla inglesa de *Distinguished Name*.

X.500 define un protocolo de acceso al servicio que permite realizar operaciones de consulta, y también operaciones de modificación de la información de los objetos. Estas últimas operaciones normalmente sólo están permitidas a ciertos usuarios autorizados, y por tanto hacen falta mecanismos de autenticación de los usuarios, y estos mecanismos están definidos en la Recomendación X.509. Hay un mecanismo básico que utiliza contraseñas, y un mecanismo más avanzado que utiliza certificados.

A continuación podéis ver la estructura de un certificado X.509, con sus campos y subcampos. En la notación usada aquí, “rep.” significa que el campo se puede repetir una o más veces, y “opc.” significa que el campo es opcional (y por tanto “opc. rep.” significa que el campo se puede repetir cero o más veces).

<u>Campo</u>	<u>Tipo</u>
toBeSigned	
version (opc.)	entero
serialNumber	entero
signature	
algorithm	identificador único
parameters	(depende del algoritmo)
issuer	DN
validity	
notBefore	fecha y hora
notAfter	fecha y hora
subject	DN
subjectPublicKeyInfo	
algorithm	
algorithm	identificador único
parameters	(depende del algoritmo)
subjectPublicKey	cadena de bits
issuerUniqueIdentifier (opc.)	cadena de bits
subjectUniqueIdentifier (opc.)	cadena de bits
extensions (opc. rep.)	
extnId	identificador único
critical (opc.)	booleano
extnValue	(depende de la extensión)
algorithmIdentifier	
algorithm	identificador único
parameters	(depende del algoritmo)
encrypted	cadena de bits

El cuerpo del certificado (“toBeSigned” en la tabla anterior) está formado por los siguientes campos:

- El campo `version` es el número de versión del formato: puede ser 1, 2 ó 3. Aunque es un campo opcional, es obligatorio si la versión es 2 ó 3 (por tanto, la ausencia de este campo indica que la versión es 1).
- El campo `serialNumber` es el número de serie del certificado, y sirve para distinguirlo de todos los demás que haya generado la misma CA.
- El campo `signature` indica el algoritmo utilizado por la CA para firmar el certificado.
- El campo `issuer` es el nombre distintivo (DN) del firmante o **emisor** del certificado, es decir, de la CA que lo ha generado.
- El campo `validity` indica el período de validez del certificado, entre un instante inicial y un instante de caducidad. Cuando hablemos más adelante de las listas de revocación, veremos la utilidad de la fecha de caducidad en los certificados.
- El campo `subject` es el nombre distintivo (DN) del **sujeto** a nombre del cual está emitido el certificado, es decir, el titular de la clave pública que figura en el certificado.
- En el campo `subjectPublicKeyInfo` se encuentra la clave pública del sujeto: a qué algoritmo corresponde, y su valor.
- Los campos `issuerUniqueIdentifier` y `subjectUniqueIdentifier` se introdujeron en la versión 2, pero no se suelen utilizar porque con el campo `extensions` son innecesarios.
- El campo `extensions` se introdujo en la versión 3, y es una lista de atributos adicionales del certificado. El subcampo `extnId` indica de qué tipo de extensión se trata. El subcampo `critical` indica si la extensión es crítica o no: si lo es, las aplicaciones que no reconozcan el tipo de extensión han de considerar el certificado como no válido (este subcampo es opcional porque por defecto las extensiones no son críticas). El subcampo `extnValue` es el valor concreto de la extensión.

Después del cuerpo hay el campo `algorithmIdentifier` (que en realidad es redundante con el campo `signature`), y finalmente hay el campo `encrypted` que es la firma del certificado, es decir, el resultado de cifrar con la clave privada de la CA el resumen (*hash*) del cuerpo del certificado.

Para calcular el resumen, el cuerpo del certificado se tiene que representar como una secuencia de bytes mediante una codificación no ambigua, que garantice que cualquiera que quiera verificar la firma pueda reconstruir exactamente la misma secuencia de bytes. Las reglas para obtener esta codificación se llaman DER, y forman parte de la notación ASN.1, que es la notación en que está definido el formato de los certificados X.509.

Números de serie

Los números de serie de los certificados generados por una CA no tienen por qué ser consecutivos: es suficiente que cada uno sea distinto a todos los anteriores.

Tipo de extensiones

Hay un cierto número de extensiones estándares, pero se pueden definir más de nuevas, según las necesidades de las aplicaciones, mientras cada una se identifique con un valor inambiguo del subcampo `extnId`.

ASN.1 y DER

ASN.1 es la sigla de *Abstract Syntax Notation One*, y DER es la sigla de *Distinguished Encoding Rules*.

1.3.2. Cadenas de certificados y jerarquías de certificación

Un certificado nos soluciona el problema de la autenticidad de la clave pública si está firmado por una CA en la cual confiamos. Pero ¿qué pasa si nos comunicamos con un usuario que tiene un certificado emitido por una CA que no conocemos?

Existe la posibilidad que una CA tenga un certificado que garantice la autenticidad de su clave pública, firmado por otra CA. Esta otra CA puede que sí que la conozcamos, o puede que a su vez tenga un certificado firmado por una tercera CA, y así sucesivamente. De esta forma, se puede establecer una jerarquía de autoridades de certificación, donde las CA de nivel más bajo emiten los certificados de usuario, y las CA de cada nivel son certificadas por una de nivel superior.

En un mundo ideal, podría haber una CA raíz que estuviera en la parte superior de la jerarquía y tuviera la máxima autoridad. En la práctica, esta CA raíz global no existe, y seguramente no existirá nunca (sería difícil que fuera aceptada por todo el mundo). En lugar de haber un sólo árbol que incluya a todas las CA del mundo, lo que existe en la realidad son árboles independientes (algunos, posiblemente con una sola CA), cada uno con su CA raíz.

Para que podamos verificar la autenticidad de su clave pública, un usuario nos puede enviar su certificado, más el certificado de la CA que lo ha emitido, más el de la CA que ha emitido este otro certificado, etc. hasta llegar al certificado de una CA raíz. Esto es lo que se denomina una **cadena de certificados**. Los certificados de las CA raíz tienen la propiedad de ser auto-firmados, es decir, están firmados por ellas mismas.

Un posible tipo de extensión de los certificados X.509 es `basicConstraints`, y un campo de su valor indica si el certificado es de CA (se puede usar su clave para emitir otros certificados) o no. En el caso de que lo sea, otro subcampo (`pathLenConstraint`) permite indicar el número máximo de niveles de jerarquía que se sitúan por debajo de esta CA.

1.3.3. Listas de revocación de certificados (CRL)

La Recomendación X.509, además de definir el formato de los certificados, también define otra estructura llamada **lista de revocación de certificados** o CRL. Una lista de este tipo sirve para publicar los certificados que han dejado de ser válidos antes de su fecha de caducidad. Los motivos pueden ser diversos: se ha emitido otro certificado que sustituye al revocado, ha cambiado el DN del titular (por ejemplo, ha dejado de trabajar en la empresa en la que estaba), le han robado su clave privada, etc.

De este modo, si queremos asegurarnos completamente de la validez de un certificado, no basta con verificar su firma, sino que debemos obtener la versión actual de la CRL (publicada por la CA que emitió el certificado) y comprobar que el certificado no aparece en esta lista.

CRL

CRL es la sigla de *Certificate Revocation List*.

Una CA normalmente actualizará su CRL de forma periódica, añadiendo cada vez los certificados que hayan sido revocados. Cuando llegue la fecha de caducidad que constaba en el certificado, ya no será necesario volver a incluirlo en la CRL. Esto permite que las CRL no crezcan indefinidamente.

Éstos son los campos de una CRL:

<u>Campo</u>	<u>Tipo</u>
toBeSigned	
version (opc.)	entero
signature	
algorithm	identificador único
parameters	(depende del algoritmo)
issuer	DN
thisUpdate	fecha y hora
nextUpdate (opc.)	fecha y hora
revokedCertificates (opc. rep.)	
userCertificate	entero
revocationDate	fecha y hora
crlEntryExtensions (opc. rep.)	
extnId	identificador único
critical (opc.)	booleano
extnValue	(depende de la extensión)
crlExtensions (opc. rep.)	
extnId	identificador único
critical (opc.)	booleano
extnValue	(depende de la extensión)
algorithmIdentifier	
algorithm	identificador único
parameters	(depende del algoritmo)
encrypted	cadena de bits

El campo `issuer` identifica la CA que emite la CRL, y en el `revokedCertificates` se encuentra la lista de los certificados revocados (cada uno identificado por su número de serie). En las extensiones de cada elemento de la lista puede haber, por ejemplo, el motivo de la revocación.

2. Sistemas de autenticación

Uno de los servicios de seguridad que se requiere en muchas aplicaciones es el de la **autenticación**. Este servicio permite garantizar que nadie ha falsificado la comunicación.

Podemos distinguir dos tipos de autenticación:

- La **autenticación de mensaje** o **autenticación de origen de datos** permite confirmar que el originador A de un mensaje es auténtico, es decir, que el mensaje no ha sido generado por un tercero Z que quiere hacer creer que lo ha generado A .

Como efecto adicional, la autenticación de mensaje proporciona implícitamente el servicio de **integridad de datos**, que permite confirmar que nadie ha modificado un mensaje enviado por A .

- La **autenticación de entidad** permite confirmar la identidad de un participante A en una comunicación, es decir, que no se trata de un tercero Z que dice ser A .

A continuación veremos cómo se puede conseguir cada uno de estos dos tipos de autenticación.

2.1. Autenticación de mensaje

Existen dos grupos de técnicas para proporcionar autenticación de mensaje:

- Los **códigos de autenticación de mensaje** o **MAC**, basados en claves simétricas.
- Las **firmas digitales**, que se basan en la criptografía de clave pública.

2.1.1. Códigos de autenticación de mensaje (MAC)

Un **código de autenticación de mensaje** o **MAC** se obtiene con un algoritmo a que tiene dos entradas: un mensaje M de longitud arbitraria, y una clave secreta k compartida por el originador y el destinatario del mensaje. Como resultado da un código $C_{\text{MAC}} = a(k, M)$ de longitud fija. El algoritmo MAC debe garantizar que sea computacionalmente inviable encontrar un mensaje $M' \neq M$ que dé el mismo código que M , y también obtener el código de un mensaje cualquiera sin conocer la clave.

MAC

MAC es la sigla de *Message Authentication Code*.

Las propiedades de los algoritmos MAC hacen que dado un par (M, C_{MAC}) un atacante no pueda obtener otro par (M', C_{MAC}) , ni en general cualquier par (M', C'_{MAC}) . Por tanto, el código MAC sirve como prueba de autenticidad del mensaje.

Un posible algoritmo MAC consiste en aplicar al mensaje M un cifrado en bloque en modo CBC con la clave k , y tomar el último bloque cifrado como código C_{MAC} . Sin embargo, los algoritmos MAC usados actualmente, suelen estar basados en una función *hash*. Por ejemplo, la técnica de calcular el resumen a partir de la concatenación del mensaje y la clave, o la de calcular el resumen del mensaje y cifrarlo con la clave, podrían servir como algoritmos MAC. Para mejorar la seguridad contra ciertos ataques, muchos protocolos usan una técnica de autenticación de mensajes un poco más sofisticada, conocida como HMAC.

2.1.2. Firmas digitales

Los códigos MAC, dado que se basan en una clave secreta, sólo tienen significado para quienes conozcan dicha clave. Si A envía mensajes a B autenticados con una clave compartida, sólo B podrá verificar la autenticidad de estos mensajes.

Por otro lado, en caso de un conflicto en que A denegase la autoría de un mensaje autenticado, B no podría demostrar delante de un tercero imparcial (por ejemplo, un árbitro o un juez) que el mensaje lo generó A . Revelar la clave secreta no sería prueba suficiente ya que, por el hecho de ser conocida por las dos partes, siempre habría la posibilidad que el mensaje en disputa y su código de autenticación los hubiera generado B .

En cambio, si A autentica los mensajes adjuntándoles la firma digital calculada con su clave privada, todo el mundo podrá verificarlos con su clave pública. Esta técnica de autenticación proporciona, como efecto adicional, el servicio de **no repudio**. Esto quiere decir que un destinatario B puede demostrar fehacientemente ante un tercero que un mensaje ha sido generado por A .

Como hemos visto en el subapartado 1.2, los algoritmos de firma digital usados normalmente se basan en el cálculo de un *hash* y en un cifrado mediante una clave privada. Son ejemplos de algoritmos de firma el RSA, el ElGamal, y el estándar DSA (*Digital Signature Algorithm*).

2.2. Autenticación de entidad

La autenticación de entidad se utiliza cuando en una comunicación una de las partes quiere asegurarse de la identidad de la otra. Normalmente, esta autenticación es un requisito para permitir el acceso a un recurso restringido, como, por ejemplo, una cuenta de usuario en un ordenador, dinero en efectivo en un cajero automático, acceso físico a una habitación, etc.

Código HMAC

Para calcular el código HMAC, al mensaje se le añade como prefijo una cadena de bits derivada de la clave, se calcula su *hash*, al resultado se le prefija otra cadena de bits derivada de la clave, y se vuelve a calcular el *hash*. (Aunque se tenga que llamar dos veces la función *hash*, la segunda será mucho más rápida puesto que los datos a resumir serán más cortos.)

En general, las técnicas utilizadas para la identificación de un usuario A pueden estar basadas en:

- Algo que A *sabe* como, por ejemplo, una contraseña o una clave privada.
- Algo que A *tiene* como, por ejemplo, una tarjeta con banda magnética o con chip.
- Algo que A *es* o, dicho de otro modo, alguna propiedad inherente a A como, por ejemplo, sus características biométricas.

La biometría es una disciplina relativamente moderna que es posible que tenga una cierta implantación en un futuro próximo. Aun así, en este módulo nos centraremos en las técnicas basadas en el intercambio de información por medios electrónicos.

Una diferencia entre la autenticación de mensaje y la autenticación de entidad es que la primera puede ser intemporal (es posible verificar la autenticidad de un documento firmado, por ejemplo, diez años atrás) mientras que la segunda normalmente se realiza en tiempo real. Esto quiere decir que para la autenticación de entidad se puede llevar a cabo un protocolo interactivo, en el que ambas partes se intercambien mensajes hasta que la identidad en cuestión quede confirmada.

A continuación veremos dos grupos de técnicas que se pueden utilizar para la autenticación de entidad:

- Las basadas en **contraseñas** (o *passwords*, en inglés), también llamadas *técnicas de autenticación débil*.
- Las basadas en **protocolos de reto-respuesta** (*challenge-response*, en inglés), también llamadas técnicas de autenticación fuerte.

2.2.1. Contraseñas

La idea básica de la autenticación basada en contraseñas es que el usuario A manda su identidad (su identificador de usuario, su nombre de *login*, etc.) seguida de una contraseña secreta x_A (una palabra o combinación de caracteres que el usuario pueda memorizar). El verificador B comprueba que la contraseña sea válida, y si lo es da por buena la identidad de A .

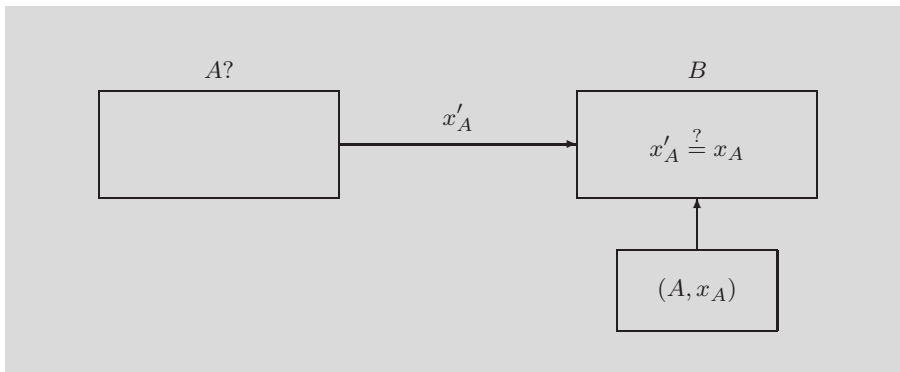
Existen distintas maneras de llevar a cabo esta autenticación, y también hay maneras diversas de intentar atacarla. A continuación veremos algunas variantes de la autenticación con contraseñas que intentan evitar determinados tipos de ataques.

Lista de contraseñas en claro

La manera más simple de comprobar una contraseña es que el verificador tenga una lista de las contraseñas asociadas a los usuarios, es decir, una lista de pares (A, x_A) . Cuando A envía su contraseña, el verificador compara directamente el valor recibido x'_A con el que figura en la lista, x_A .

Técnicas biométricas

Las técnicas biométricas hacen uso de características fisiológicas humanas (como la huella dactilar, el iris, la retina, la cara o la mano) o características del comportamiento humano (como el habla, la firma manual o la pulsación de teclas).



Si se trata de las contraseñas correspondientes a usuarios de un sistema informático, es evidente que no pueden estar guardadas en un fichero de acceso público: es necesario que la lista esté protegida contra lectura. Pero si alguien encuentra la manera de saltarse esta protección (cosa que en ocasiones pasa en los sistemas multiusuario), automáticamente tendrá acceso a las contraseñas de todos los usuarios.

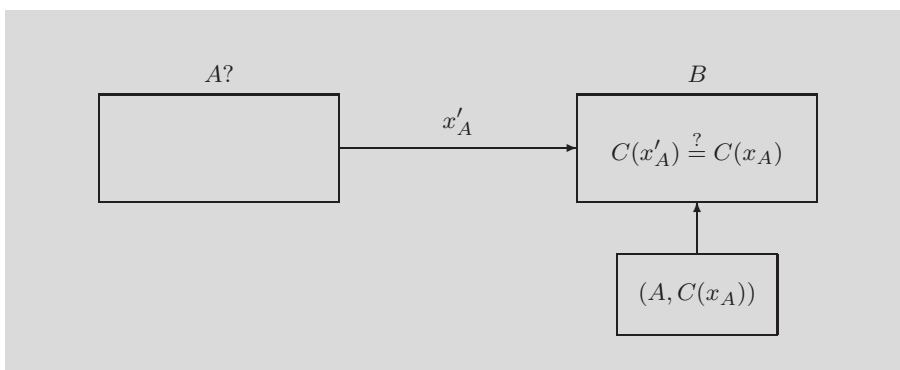
Además, en estos sistemas normalmente hay un usuario administrador o “super-usuario” que tiene acceso a todos los ficheros, y por tanto a las contraseñas de los usuarios. Un super-usuario que fuera mal intencionado podría hacer un mal uso de este privilegio. O un super-usuario que tuviera un momento de descuido podría dar pie a que otro usuario se aprovechara.

Lista de contraseñas codificadas

Una segunda opción consiste en que en la lista de contraseñas, en lugar de estar guardadas éstas en claro por pares (A, x_A) , cada una esté codificada con alguna transformación C de manera que no se pueda deducir su valor real. De este modo, la lista debe contener pares $(A, C(x_A))$.

Esta codificación C podría ser, por ejemplo, un cifrado, pero entonces sería preciso tomar las precauciones oportunas con la clave de descifrado (quien consiguiera acceder a esta clave podría obtener todas las contraseñas). Lo más habitual es que la codificación consista en la aplicación de una función unidireccional como, por ejemplo, una función *hash*.

Para la verificación, B debe calcular el valor transformado $C(x'_A)$ a partir de la contraseña recibida, y compararlo con el que consta en la lista.



Codificación de contraseñas en Unix

Un caso muy conocido es el de la gestión de las contraseñas en el sistema operativo Unix, ya que la mayoría de las variantes de Unix usadas actualmente utilizan técnicas muy parecidas, basadas en las versiones originales de este sistema.

En Unix existe un fichero en el que se guardan las contraseñas de los usuarios codificadas con una función unidireccional. Pero en lugar de una función *hash*, lo que se utiliza es un cifrado simétrico: el texto en claro es fijo (una cadena de bits todos iguales a 0), como clave de cifrado se utiliza la contraseña, y lo que se guarda en la lista es el texto cifrado. De esta forma se aprovecha la propiedad que tienen los algoritmos criptográficos que no permiten deducir la clave a partir del texto cifrado ni que se conozca el texto en claro.

Dado que la codificación de las contraseñas es unidireccional, nadie (ni tan siquiera el superusuario) puede saber cuál es la contraseña de cada usuario aunque tenga acceso a la lista. Esto haría innecesaria la protección contra lectura del fichero en el que hay la lista.

Pero, aunque la codificación no sea reversible, puede ser vulnerable a otro tipo de ataque: la fuerza bruta. Si el atacante conoce la contraseña codificada $C(x_A)$ y conoce el algoritmo de codificación C , puede probar de codificar posibles contraseñas hasta encontrar un resultado que coincida con $C(x_A)$.

Existe un hecho que favorece al atacante en este caso: si los usuarios pueden escoger sus contraseñas, normalmente no usarán combinaciones arbitrarias de caracteres sino palabras fáciles de recordar. Por tanto, el espacio de búsqueda se reduce considerablemente.

El atacante puede, pues, limitarse a probar palabras de un diccionario o combinaciones derivadas a partir de estas palabras. Por este motivo a este tipo de ataque se lo conoce como **ataque de diccionario**.

Programas “rompe-contraseñas”

Desde hace años existen los programas “rompe-contraseñas”, o “*password crackers*” en inglés. A un programa de este tipo se le da una lista de contraseñas codificadas y un diccionario, y va probando cada una de las palabras, tanto directamente como con diferentes variaciones: todo minúsculas, todo mayúsculas, la primera letra mayúscula, con las letras al revés, añadiendo una cifra numérica, añadiendo dos, cambiando ciertas letras por cifras, etc. También puede probar combinaciones con datos adicionales como los identificadores de los usuarios, sus nombres, apellido, etc.

Es sorprendente la cantidad de contraseñas que estos programas pueden llegar a encontrar en sólo unas pocas horas.

Espacio de contraseñas

Si consideramos las posibles combinaciones de, por ejemplo, 8 caracteres (suponiendo cada carácter comprendido entre los códigos ASCII 32 y 126, es decir, 95 caracteres distintos), existen 95^8 (aprox. $6,6 \cdot 10^{15}$). En cambio, hay estudios que indican que una gran parte de las contraseñas que escogen los usuarios se pueden encontrar en un diccionario de 150000 palabras, un espacio 10^{10} veces menor.

Técnicas para dificultar los ataques de diccionario

A continuación veremos algunas posibles técnicas para dificultar los ataques de diccionario.

1) Ocultar la lista de contraseñas codificadas

Una primera solución es restringir el acceso a la lista de contraseñas, protegiéndola contra lectura. Aunque en la lista aparezcan las contraseñas codificadas, el acceso a esta lista por parte de un atacante le permite realizar cómodamente un ataque de diccionario.

Lista de contraseñas codificadas en Unix

En las versiones antiguas del sistema Unix, las contraseñas codificadas estaban en el fichero `/etc/passwd`. Este mismo fichero se aprovechaba para guardar otros datos sobre la cuenta de cada usuario: su directorio inicial, su *shell*, el nombre del usuario, etc. Dado que esta información es de acceso público, el fichero `/etc/passwd` tenía permiso de lectura para todos los usuarios.

En las versiones modernas de Unix continúa existiendo un fichero `/etc/passwd` de lectura pública con información sobre las cuentas de los usuarios, pero las contraseñas codificadas ya no se guardan en este fichero sino en otro, `/etc/shadow`, sobre el cual ningún usuario (excepto el super-usuario) tiene permiso de lectura.

Si el atacante no tiene acceso a la lista de contraseñas, ya no podrá realizar un ataque “fuera de línea” (*off-line*), es decir, un ataque realizado en un ordenador escogido por el atacante a su conveniencia (por ejemplo, un ordenador potente, donde nadie pueda ver que está haciendo, etc.). No le quedará más remedio que realizar el ataque “en línea” (*on-line*), es decir, directamente con el verificador real (por ejemplo, el programa `login` del sistema que quiere atacar), enviándole contraseñas para ver si las acepta o no.

Si el verificador ve que alguien está realizando pruebas de autenticación fallidas, esto es una señal de que puede tratarse de un ataque en línea. Entonces puede tomar ciertas medidas para contrarrestar este ataque. Por ejemplo, se puede hacer que si se recibe un cierto número de contraseñas inválidas consecutivas, el recurso asociado quede bloqueado.

Protección contra ataques en las tarjetas con PIN

El mecanismo del bloqueo se utiliza típicamente en los métodos de autenticación basados en dispositivos físicos, como las tarjetas bancarias o los módulos SIM (*Subscriber Identity Module*) de los teléfonos móviles. Estos dispositivos requieren que el usuario introduzca un PIN (*Personal Identification Number*), que hace las funciones de contraseña.

Por motivos históricos y de conveniencia de los usuarios, este PIN es un número de longitud corta (por ejemplo de cuatro cifras). En este caso, para evitar los ataques de fuerza bruta (que sólo necesitarían 10.000 intentos como máximo), el dispositivo se bloquea, por ejemplo, al tercer intento equivocado.

La protección que proporciona el bloqueo puede ser un inconveniente para el usuario legítimo que no tiene ninguna culpa de que alguien haya intentando descubrir su contraseña. Para evitarle este inconveniente, una posibilidad es que cada usuario tenga asociada otra contraseña de desbloqueo, mucho más difícil de adivinar (por ejemplo, un PIN de ocho cifras para desbloquear el PIN de cuatro cifras).

Otra posibilidad consiste en permitir múltiples intentos de autenticación, pero en lugar de responder inmediatamente con un mensaje de “contraseña incorrecta”, demorar esta respuesta con un tiempo de retraso, que irá creciendo a medida que se vayan enviando más contraseñas erróneas por parte del mismo usuario. Esto ralentizaría tanto un ataque que lo haría inviable a partir de unos pocos intentos. Y cuando el usuario legítimo utilice su contraseña correcta, el tiempo de respuesta será el normal.

2) Reglas para evitar contraseñas fáciles

La solución de ocultar la lista de contraseñas codificadas da una seguridad parecida a la de la lista de contraseñas en claro. Si alguien descubre la lista (saltándose la protección contra lectura), puede realizar sin más problemas un ataque de diccionario.

Otro modo de dificultar este ataque es obligar a los usuarios a escoger contraseñas que cumplan unas determinadas reglas para que no sean fáciles de adivinar. Por ejemplo:

- Que la contraseña tenga una longitud mínima.
- Que no sea todo letras ni todo números.
- Que las letras no coincidan con ninguna palabra de diccionario ni con combinaciones triviales de las palabras (como escribir las letras al revés, etc.).
- Que la contraseña no se derive del identificador del usuario, de su nombre, apellido, etc.
- Etc.

Con estas reglas se consigue que el espacio de contraseñas donde hacer la búsqueda sea más grande del que es habitual, y el ataque de diccionario necesite más tiempo.

Por otro lado, también es cierto que cuanto más tiempo mantenga un usuario su contraseña sin cambiar, más grande será la probabilidad que un atacante consiga descubrirla. Por esto es recomendable que los usuarios renueven las contraseñas periódicamente. Hay sistemas que implantan una política de cambio forzado de contraseñas. A partir de un cierto período (por ejemplo, 90 días) la contraseña se considera caducada y el usuario la tiene que cambiar por una de nueva.

3) Añadir complejidad a la codificación de las contraseñas

Otra solución para dificultar los ataques de diccionario es ralentizarlos haciendo que cada contraseña cueste más de codificar. Por ejemplo, si el algoritmo de codificación no es directamente una función *hash* sino un bucle de N llamadas a la función *hash*, probar cada contraseña cuesta N veces más.

El valor N se puede escoger tan grande como se quiera, pero teniendo en cuenta que si es demasiado grande los usuarios legítimos también podrán notar que la autenticación normal es más lenta.

Complejidad de la codificación de las contraseñas en Unix

La solución que se adoptó en Unix fue repetir N veces el cifrado para obtener la contraseña codificada, y se escogió el valor $N = 25$.

Por tanto, aunque el atacante disponga de una implementación rápida del algoritmo de cifrado, el tiempo esperado para romper una contraseña se multiplica por 25.

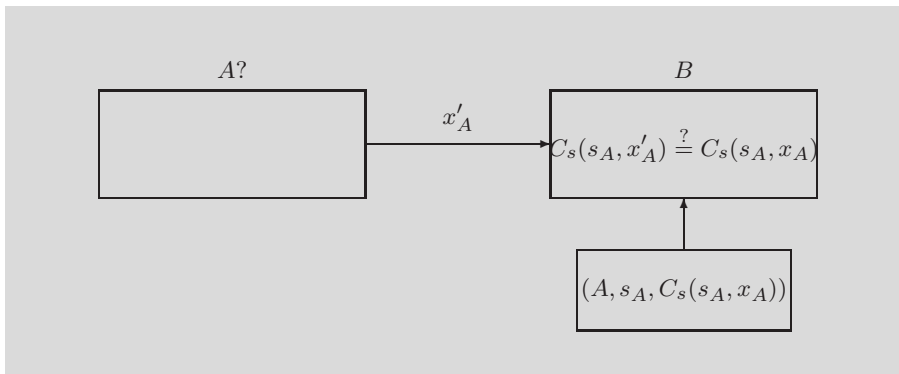
4) Añadir bits de sal a la codificación de las contraseñas

En el subapartado 1.1 hemos visto que, para variar el resultado del cifrado aunque se utilice la misma clave con el mismo texto en claro, hay la posibilidad de introducir los llamados “bits de sal” para modificar la clave. De la misma manera, se puede definir un algoritmo de codificación C_s que, a más de la contraseña, tenga como entrada unos bits de sal.

De este modo, cada vez que un usuario A cambie su contraseña x_A , se generan aleatoriamente los bits de sal s_A y se guardan estos bits juntamente con la contraseña codificada $C_s(s_A, x_A)$. A la hora de verificar, se vuelve a calcular este valor a partir de los mismos bits de sal y se compara con el valor guardado.

Caducidad de las contraseñas

Un tiempo de caducidad de las contraseñas demasiado corto también podría ser contraproducente, porque si los usuarios deben pensar muchas contraseñas en poco tiempo, pueden acabar olvidando cuál era la última, o anotándola en un papel, con lo cual la contraseña es mucho más vulnerable.



Esta técnica no complica la verificación de una contraseña ni un ataque de diccionario contra esta contraseña específica. Pero si el ataque de diccionario se realiza sobre distintas contraseñas a la vez (por ejemplo, sobre la lista entera de todos los usuarios, como hacen normalmente los programas rompe-contraseñas), el tiempo de cálculo se multiplica por el número total de contraseñas, suponiendo que todas estén codificadas con bits de sal distintos.

Esto es así porque, sin bits de sal, se puede tomar cada palabra del diccionario, codificarla, y comparar el resultado con todas las contraseñas codificadas. En cambio, si hay bits de sal, es preciso realizar una codificación separada para cada contraseña, cada una con sus bits de sal correspondientes.

Con los bits de sal también se evita una variante más eficiente del ataque de diccionario, en la cual el atacante no va probando una a una las palabras, sino que utiliza una lista preconstruída con las palabras del diccionario ya codificadas. Si, por ejemplo, el diccionario contiene 150.000 palabras, cuando no se utilizan bits de sal el atacante sólo tiene que buscar la contraseña codificada en una lista de 150.000 palabras codificadas. Pero si se utilizan 12 bits de sal, el tamaño de la lista se multiplicaría por más de 4.000 ($2^{12} = 4,096$), y el atacante debería trabajar con una lista de más de seiscientos millones de palabras codificadas.

Otra ventaja de los bits de sal es que si, por casualidad, dos usuarios eligen la misma contraseña, los valores codificados de sus contraseñas serán distintos (si los bits de sal también lo son), y aunque vieran la lista de contraseñas, no sabrían que ambos tienen la misma.

Bits de sal en las contraseñas en Unix

En Unix se utilizan precisamente 12 bits aleatorios de sal (que se obtienen a partir de los bits de menos peso de la hora en el momento en que el usuario se cambia la contraseña). Esto significa que hay 4.096 valores posibles de estos bits. Una lista completa de contraseñas codificadas a partir de un diccionario de 150.000 palabras tendría que contener, pues, más de seiscientos millones de entradas. Esto podría ser una cantidad descomunal en la época en que se desarrolló el sistema Unix, pero actualmente una lista de estas dimensiones cabe perfectamente en el disco duro de cualquier ordenador personal.

Por otro lado, el algoritmo de cifrado que se utiliza en Unix para obtener la codificación de las contraseñas es el DES. Como hemos visto antes, para codificar una contraseña se aplica un cifrado a un texto formado por todos los bits a 0, y se usa la contraseña como clave de cifrado (esto explica por qué las versiones estándar de Unix sólo utilizan contraseñas de hasta 8 caracteres). En este caso, los bits de sal no se utilizan para modificar la clave, sino que sirven para permutar ciertos bits de los resultados intermedios que se obtienen en cada iteración del algoritmo DES.

Con esto se consigue otro objetivo: que un atacante no pueda utilizar directamente una implementación eficiente del algoritmo DES (las más eficientes son los chips DES, es decir, las implementaciones con hardware), sino que tenga que introducir modificaciones (si se trata de un chip DES, son precisos ciertos recursos que no siempre están al alcance de cualquiera).

5) Uso de *passphrases*

La propiedad que aprovechan los ataques de diccionario es que el conjunto de contraseñas que utilizan normalmente los usuarios es un subconjunto muy pequeño de todo el espacio de contraseñas posibles.

Para ampliar este espacio se puede hacer que el usuario no utilice palabras relativamente cortas, de unos 8 caracteres, sino frases más largas, llamadas "*passphrases*". La codificación de estas *passphrases* puede continuar siendo una función unidireccional, como por ejemplo una función *hash* (con la misma longitud que en el caso de las contraseñas). La diferencia es que, si antes la lista de contraseñas codificadas contenía valores de entre un total de unos 150000 distintos, con *passphrases* contendrá muchísimos más valores posibles, y la búsqueda exhaustiva del ataque de diccionario será mucho más larga.

Contraseñas de un solo uso

Todos los esquemas de autenticación con contraseñas que hemos visto hasta ahora, aparte de los ataques de diccionario, son vulnerables a los **ataques de repetición** o de "*replay*". Para llevar a cabo este ataque, hace falta interceptar la comunicación entre *A* y *B* durante una autenticación, por ejemplo utilizando un *sniffer*, y ver cuál es el valor x_A enviado. Entonces el atacante *Z* sólo tiene que realizar una autenticación delante *B* enviándole este mismo valor x_A , y *B* se pensará que se trata del auténtico usuario *A*.

Este tipo de ataque se evita con los protocolos de autenticación fuerte que veremos a continuación. Pero existe otra técnica que, aunque se puede considerar basada en contraseñas, tiene algunas propiedades de la autenticación fuerte, entre ellas la resistencia a los ataques de repetición. Esta técnica es la de las llamadas **contraseñas de un solo uso** ("*one-time passwords*").

Como su nombre indica, las contraseñas de un solo uso son contraseñas que sólo son válidas una vez, y a la vez siguiente es preciso utilizar una contraseña distinta. Así, aunque el atacante vea qué valor se está enviando para la autenticación, no le servirá de nada porque ya no lo podrá volver a utilizar.

Existen distintas posibilidades para implementar la autenticación con contraseñas de un solo uso. Por ejemplo, podemos considerar las siguientes:

- Lista de contraseñas compartida: *A* y *B* se ponen de acuerdo, de manera segura (es decir, no a través de un canal que pueda ser interceptado), en una lista de *N* contraseñas. Si se trata de una lista ordenada, cada vez que *A* se tenga que autenticar utilizará la siguiente contraseña de la lista. Alternativamente, las contraseñas pueden tener asociado un identificador: en cada autenticación *B* escoge una contraseña de las que aún no

se hayan utilizado, envía a A su identificador, y A debe responder con la contraseña correspondiente.

- Contraseñas basadas en una función unidireccional (normalmente una función *hash*):
 A escoge un valor secreto x_0 y utiliza una función unidireccional h para calcular los siguientes valores:

$$\begin{aligned}x_1 &= h(x_0) \\x_2 &= h(x_1) = h^2(x_0) \\&\vdots \\x_N &= h(x_{N-1}) = h^N(x_0)\end{aligned}$$

y envía el valor x_N a B (B debe asegurarse de que ha recibido este valor del usuario A auténtico).

Entonces A inicializa un contador y a N . Cada vez que se tenga que autenticar, A enviará el valor x_{y-1} , y B aplicará la función unidireccional a este valor, lo comparará con el que tiene guardado, y comprobará si $h(x_{y-1}) = x_y$. Si se cumple esta igualdad, la contraseña es válida. Para la siguiente vez, B sustituye el valor que tenía guardado, x_y , por el que acaba de recibir, x_{y-1} , y A actualiza el contador y disminuyéndolo en 1.

Dado que la función h es unidireccional, nadie que vea el valor x_{y-1} sabrá cuál es el valor que corresponderá la próxima vez (x_{y-2}), ya que A obtuvo x_{y-1} como $h(x_{y-2})$, y hacer el cálculo inverso (es decir, obtener x_{y-2} a partir de x_{y-1}) debe ser computacionalmente inviable.

La técnica de las contraseñas basadas en una función *hash* es más eficiente que la de la lista compartida, porque B sólo debe recordar el último valor x_y recibido.

Implementaciones de las contraseñas de un solo uso

Existen varias implementaciones de contraseñas basadas en una función *hash* como, por ejemplo, **S/KEY** o **OPIE**. Con estos programas, cuando el usuario A está conectado de manera segura (por ejemplo localmente, es decir, frente a la consola) al sistema servidor con el que se quiere autenticar, puede generar una lista de N contraseñas, imprimirla, y llevarla encima cada vez que deba realizar una autenticación desde un sistema remoto. Para facilitar la introducción de las contraseñas, el programa convierte los bits en una secuencia de palabras cortas. Por ejemplo:

```
...
90: BAND RISE LOWE OVEN ADEN CURB
91: JUTE WONT MEEK GIFT OWL PEG
92: KNOB QUOD PAW SEAM FEUD LANE
...
```

La traducción se hace a partir de una lista de 2048 palabras de hasta 4 caracteres, y a cada una se le asigna una combinación de 11 bits.

El usuario A no necesita recordar el valor del contador y , ya que este valor se guarda en el servidor. A cada autenticación, el servidor envía el valor actual del contador, el usuario lo utiliza para consultar su lista, y responde con la contraseña correspondiente. Para la siguiente vez, el servidor actualizará el contador decrementándolo en 1.

2.2.2. Protocolos de reto-respuesta

El problema que tienen los esquemas de autenticación basados en contraseñas es que cada vez que se quiere realizar la autenticación se tiene que enviar el mismo valor al verificador (excepto en las contraseñas de un solo uso, como acabamos de ver). Cualquier atacante que consiga interceptar este valor fijo podrá suplantar la identidad del usuario a quien corresponda la contraseña.

Hay otro grupo de mecanismos donde el valor que se envía para la autenticación no es fijo, sino que depende de otro, generado por el verificador. Este último valor se llama **reto**, y se debe enviar al usuario *A* como primer paso para su autenticación. Entonces *A*, haciendo uso de una clave secreta, calcula una **respuesta** a partir de este reto, y lo envía al verificador *B*. Por este motivo, estos mecanismos de autenticación reciben el nombre de **protocolos de reto-respuesta**.

El algoritmo para calcular la respuesta debe garantizar que no se pueda obtener sin saber la clave secreta. Esto permite al verificador confirmar que la respuesta sólo ha podido enviarla *A*. Si se utiliza un reto distinto cada vez, un atacante no podrá sacar provecho de la información que descubra interceptando la comunicación.

Dependiendo del protocolo, el verificador puede generar los retos de diversas maneras:

- Secuencialmente: en este caso el reto es simplemente un número que se va incrementando cada vez (lo más normal es incrementarlo de uno en uno), y que por tanto no se repetirá nunca.
- Aleatoriamente: el reto puede ser generado con un algoritmo pseudoaleatorio, pero la propiedad que tiene en este caso es que no es predecible para los atacantes.
- Cronológicamente: el reto se obtiene a partir de la fecha y hora actuales (con la precisión que sea adecuada para el protocolo). Este tipo de reto también se llama **marca de hora** o “*timestamp*”. El receptor, es decir quien quiere validar la identidad, puede utilizar la marca de hora para saber si se trata de una nueva autenticación o si alguien quiere reutilizar los mensajes de otra autenticación para intentar un ataque de repetición.

La ventaja de los retos cronológicos es que, para obtenerlos, sólo es necesario un reloj, que seguramente estará disponible en cualquier sistema, mientras que con los secuenciales y los aleatorios es preciso mantener información de estado (el número de secuencia o la entrada para calcular el siguiente número pseudoaleatorio) para evitar repeticiones. El inconveniente es que es precisa una cierta sincronía entre los relojes. Si se utilizan técnicas como un servidor de tiempo que da la hora actual, también es preciso verificar que la hora obtenida sea auténtica, es decir, que un atacante no nos está engañando haciéndonos creer que es la hora que le interesa.

Dispositivos para el cálculo de las respuestas

El cálculo de la respuesta se puede hacer con algún software diseñado a tal efecto, o también se puede utilizar un dispositivo, parecido a una pequeña calculadora, que guarda la clave secreta. El usuario introduce el reto mediante el teclado de esta calculadora, y en la pantalla aparece la respuesta.

Sincronía de relojes

Si el reto se obtiene a partir de un reloj, dependiendo del protocolo puede ser necesario que emisor y receptor tengan los relojes sincronizados, o bien que se permita una cierta tolerancia entre la hora que ha enviado el emisor y la que indica el reloj del receptor. La tolerancia tiene que ser tal que permita diferencias entre los relojes pero que no dé tiempo a un tercero a realizar un ataque de repetición.

Alternativamente, si el reto es cronológico, el dispositivo no tiene teclado sino que va mostrando por la pantalla las respuestas en función de la hora actual (por ejemplo, cada minuto), en sincronía aproximada con el reloj del verificador.

Para más seguridad, el dispositivo también puede estar protegido con un PIN.

Los protocolos de reto-respuesta se pueden clasificar en dos grupos:

- Los basados en técnicas simétricas, en las cuales la clave secreta es compartida por el usuario A y el verificador B .
- Los basados en técnicas de clave pública, en las cuales A utiliza una clave privada para calcular la respuesta.

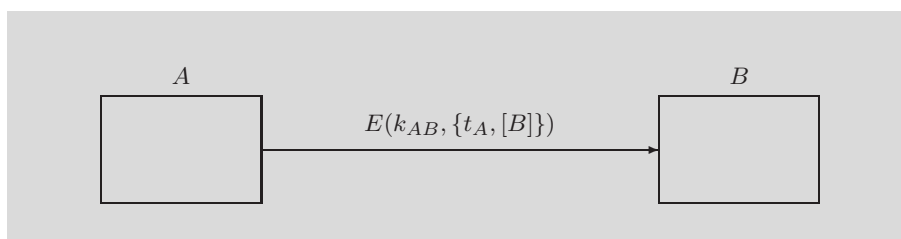
En la descripción de los protocolos que veremos a continuación, utilizaremos la notación $\{a, b, \dots\}$ para representar un mensaje que contiene los componentes $a, b, \dots$, y si alguno de estos componentes es opcional, lo representaremos entre corchetes como, por ejemplo, $\{a, [b]\}$. 

Protocolos de reto-respuesta con clave simétrica

Si el protocolo de reto-respuesta se basa en una clave k_{AB} compartida por A y B , existen distintas formas de obtener esta clave. Por ejemplo, la pueden haber acordado directamente A y B de forma segura, o bien la pueden solicitar a un servidor de claves centralizado.

1) Autenticación con marca de tiempo

El protocolo más simple es el que utiliza como reto implícito la hora actual (no es preciso el envío del reto). El usuario A obtiene una marca de tiempo t_A y la manda al verificador B cifrada con la clave compartida.



El verificador descifra el mensaje recibido y comprueba si la marca de tiempo es aceptable, es decir, si está dentro de la tolerancia de sincronía y no está repetida.

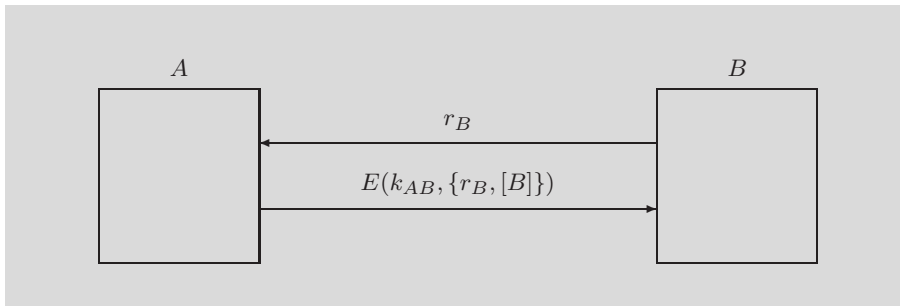
Si se utiliza la misma clave en los dos sentidos (de A a B y de B a A), el hecho de incluir la identidad del verificador B evita que, en un proceso de autenticación mutua, un atacante intente hacerse pasar por B delante de A repitiendo el mensaje en sentido contrario.

2) Autenticación con números aleatorios

En este caso es preciso enviar explícitamente el reto, que consiste en un número aleatorio r_B generado por B . La respuesta es el reto cifrado con la clave compartida.

Detección de repeticiones

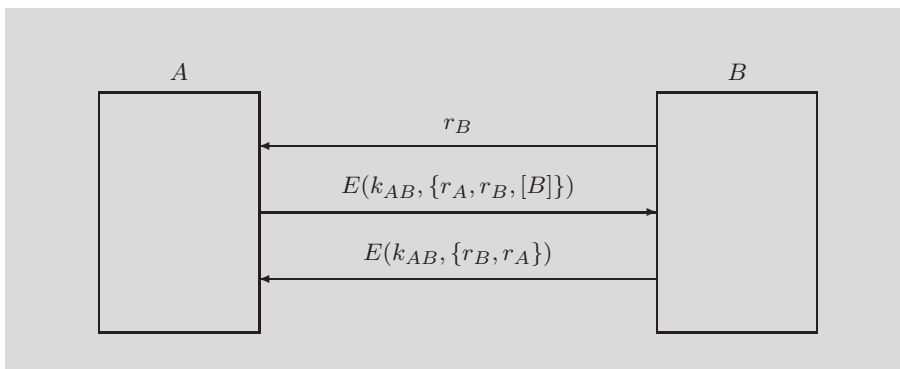
Para saber si se está reutilizando una marca de tiempo, el verificador necesita recordar cuáles se han utilizado ya, pero sólo las que estén dentro del intervalo de tolerancia. Pasado este intervalo ya se puede borrar la marca de la lista.



El verificador descifra el mensaje y comprueba que el número aleatorio que contiene es igual al reto. El hecho de incluir la identidad del verificador B evita que el mensaje se pueda utilizar en sentido contrario si alguna vez A genera como reto el mismo número r_B .

3) Autenticación mutua con números aleatorios

Con tres mensajes, el protocolo anterior se puede convertir en un protocolo de autenticación mutua, es decir, de A delante B y de B delante A . En este caso, A debe generar otro número aleatorio r_A , que actuará como reto en sentido inverso, e incluirlo en su respuesta.



Cuando B descifra la respuesta, además de comprobar que el valor r_B sea el esperado, también obtiene r_A , que deberá utilizar para enviar su respuesta a A . Entonces A comprueba que los dos números aleatorios r_A y r_B tengan los valores correctos.

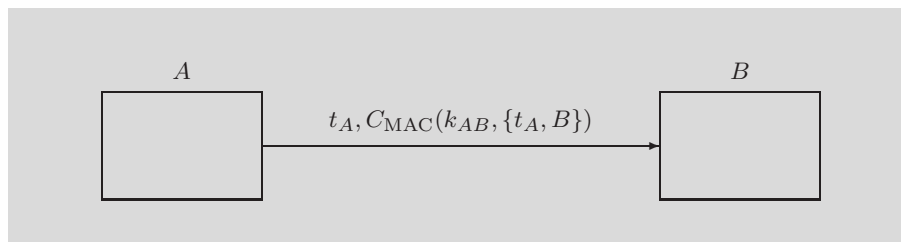
Como sucedía anteriormente, la inclusión del identificador de B en el segundo mensaje evita ataques de repetición en sentido contrario.

4) Autenticación con función unidireccional

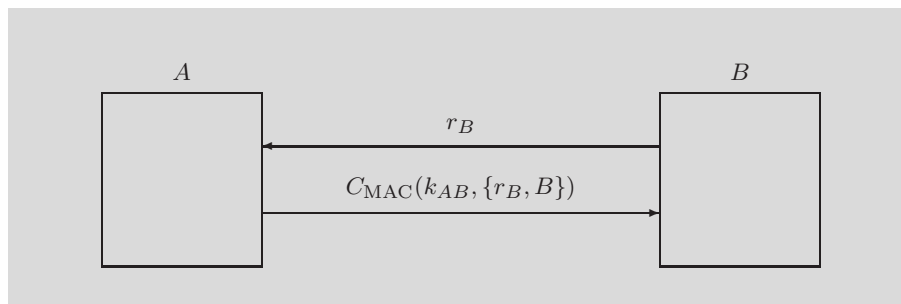
Los protocolos anteriores hacen uso de un algoritmo de cifrado simétrico, pero también es posible utilizar funciones unidireccionales con clave secreta como, por ejemplo, los algoritmos de MAC. Para la verificación, en lugar de descifrar es preciso comprobar que el MAC sea correcto y, por tanto, los valores necesarios para calcularlo se han de enviar en claro.

A continuación se muestran las variantes de los protocolos anteriores cuando se utiliza un algoritmo de MAC en lugar de un algoritmo de cifrado.

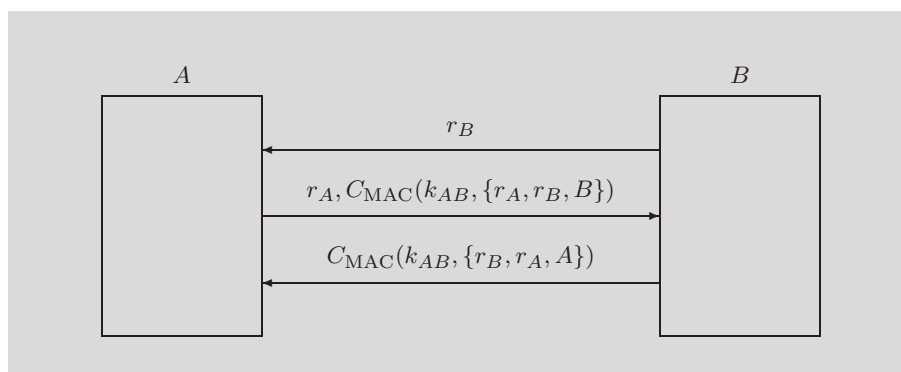
- Autenticación con marca de tiempo:



- Autenticación con números aleatorios:



- Autenticación mutua con números aleatorios:



En este caso, se debe añadir el identificador del destinatario A en el cálculo del último mensaje para evitar ataques de repetición en sentido contrario, ya que ahora los atacantes pueden ver el valor r_A .

Protocolos de reto-respuesta con clave pública

Hay dos formas de utilizar las técnicas de clave pública en los protocolos de reto-respuesta:

- El reto se manda cifrado con clave pública y la respuesta es el reto descifrado con la correspondiente clave privada.
- El reto se envía en claro y la respuesta es la firma del reto.

Dado que el usuario A tiene que utilizar su clave privada sobre un mensaje que le han enviado, ha de tomar ciertas precauciones para evitar que la otra parte haga un uso ilegítimo de la respuesta. Esto lo veremos en cada uno de los dos tipos de protocolos de reto-respuesta con clave pública.

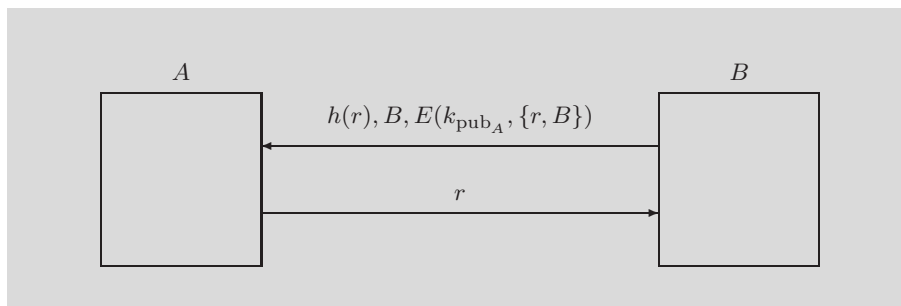
1) Descifrado del reto

Si A recibe un reto cifrado con su clave pública, antes de enviar la respuesta debe asegurarse que quien ha enviado el reto conoce su valor. Si no, un atacante podría enviarle a A un reto cifrado, haciendo creer que lo ha generado aleatoriamente, pero que en realidad lo ha extraído de otro mensaje confidencial que alguien había enviado a A cifrado con su clave pública. Si A responde a este reto, está dando al atacante el mensaje confidencial descifrado.

Una posibilidad es que A reciba, juntamente con el reto r cifrado, un resumen o *hash* de este reto.

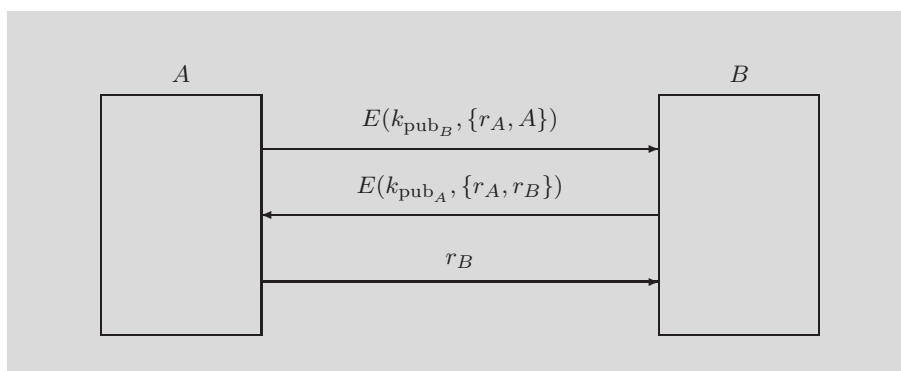
Claves para aplicaciones distintas

Para evitar los abusos que se pueden cometer contra una clave pública, es muy recomendable utilizar claves públicas distintas para cada aplicación, por ejemplo una clave para recibir mensajes confidenciales y otra para la autenticación.



Cuando A recibe el mensaje, utiliza su clave privada para obtener r y B , y comprueba que estos valores sean correctos. Si el *hash* del reto r coincide con el valor recibido $h(r)$, esto significa que B conoce este reto. Por tanto, se puede enviar a B como respuesta el valor descifrado. Si no, no se le envía nada, porque puede tratarse de un defraudador que quiera obtener ilegítimamente este valor descifrado.

Otra posibilidad es que A pueda escoger una parte del reto. Este es el principio del llamado *protocolo modificado de clave pública* de Needham-Schroeder, que además proporciona autenticación mutua.



Para A , el reto está formado por r_A y r_B : el hecho de que la primera parte la haya generado A evita que el reto haya sido elegido maliciosamente por B . Para B , el reto incluye el identificador de A , con lo cual tampoco puede ser un mensaje cifrado que un tercero había enviado a B y que A quiere descifrar ilegítimamente.

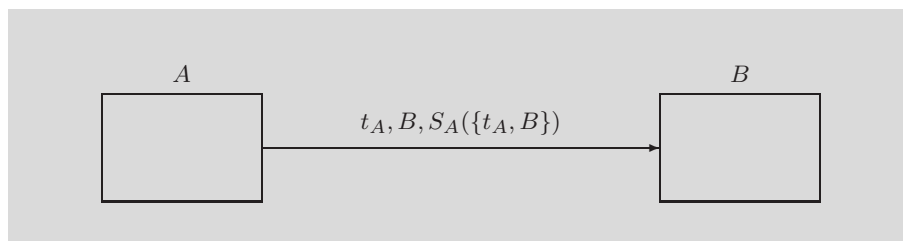
2) Firma del reto

La Recomendación X.509 no sólo especifica los formatos de certificados y listas de revocación que hemos visto en el subapartado 1.3, sino que también define unos protocolos de autenticación fuerte basados en firmas digitales que utilizan marcas de tiempo y números aleatorios. Aquí mostraremos otros protocolos, que son los equivalentes a los que hemos visto anteriormente basados en claves simétricas.

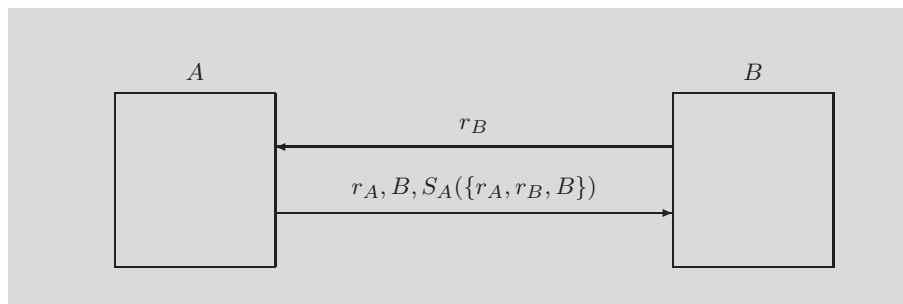
Como en el caso del descifrado con clave privada, A debe prestar atención con lo que firma: nunca debe firmar directamente un reto que le hayan enviado, sino que en la firma siempre tiene que intervenir como mínimo una parte del texto que haya escogido el propio A .

En la descripción de estos protocolos, $S_A(M)$ quiere decir la firma digital del mensaje M con la clave privada de A .

- Autenticación con marca de tiempo:

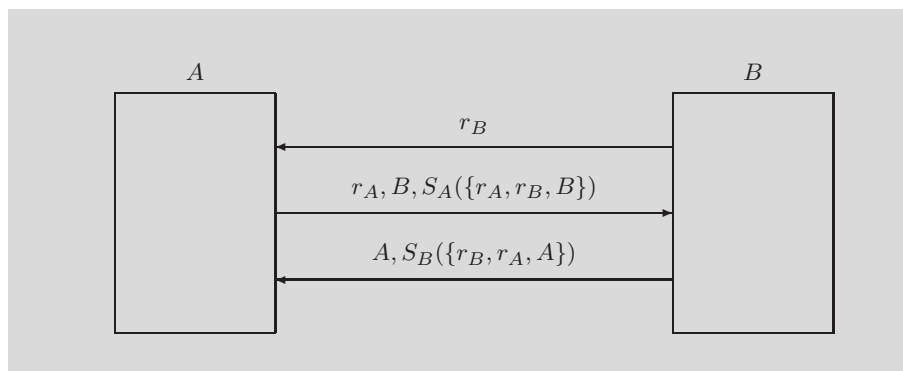


- Autenticación con números aleatorios:



Aquí la inclusión del valor r_A evita que B haya escogido r_B maliciosamente para hacer firmar a A un mensaje sin que se dé cuenta.

- Autenticación mutua con números aleatorios:



Resumen

En este módulo hemos visto que las **técnicas criptográficas** permiten cifrar un texto mediante una **clave de cifrado**, y solamente quien conozca la **clave de descifrado** correspondiente será capaz de obtener el texto original.

Según la relación que haya entre las dos claves, los algoritmos criptográficos se clasifican en **algoritmos simétricos** si la clave de cifrado y la de descifrado son la misma, o **algoritmos de clave pública** si las claves son distintas. Los algoritmos simétricos, a su vez, se pueden clasificar en **algoritmos de cifrado en flujo**, si el cifrado consiste en añadir al texto datos pseudoaleatorios calculados a partir de la clave, o **algoritmos de cifrado en bloque**, si el cifrado se realiza sobre bloques de medida fija del texto original.

La particularidad de la criptografía de clave pública es que a partir de la **clave pública** es prácticamente imposible deducir la **clave privada**. Esto permite que cualquiera que conozca la clave pública de un usuario pueda usarla para cifrar datos confidenciales, con la seguridad que solamente quien tenga la clave privada correspondiente podrá descifrarlos, y sin necesidad de acordar ninguna clave secreta a través de un canal seguro. El uso de las claves al revés (la privada para cifrar y la pública para descifrar) es la base de las **firmas digitales**.

Dado que la criptografía de clave pública es computacionalmente más costosa que la simétrica, no se utiliza nunca directamente para obtener confidencialidad, sino siempre a través de una **clave de sesión** simétrica. Del mismo modo, la firma de un texto no se calcula directamente a partir del texto, sino aplicándole una **función hash segura**. La propiedad de este tipo de función es que es muy difícil encontrar un mensaje que dé el mismo *hash* que otro.

Para garantizar que las claves públicas son auténticas, y pertenecen a quien se supone que han de pertenecer, se pueden utilizar **certificados digitales** o de clave pública, como por ejemplo los certificados X.509. Cuando una **autoridad de certificación** (CA) firma un certificado, está dando fe de la autenticidad del vínculo entre la clave pública correspondiente y la identidad del usuario. Los certificados son un componente básico de la **infraestructura de clave pública** (PKI), como también lo son las **listas de revocación de certificados** (CRL).

Las firmas digitales proporcionan el servicio de **autenticación de mensaje**. Los llamados **códigos MAC** también proporcionan este servicio, pero utilizando claves secretas compartidas en lugar de claves públicas.

Otro servicio de autenticación es el de **autenticación de entidad**. Este mecanismo permite comprobar que la otra parte de la comunicación es quien dice ser, y no un impostor. Esto

se puede conseguir con técnicas de **autenticación débil** basadas en contraseñas o, si es necesario, con técnicas de **autenticación fuerte** basadas en **protocolos de reto-respuesta**, que a diferencia de las anteriores son resistentes a muchos más ataques que las primeras.

Glosario

Ataque: Acción realizada por una tercera parte, distinta del emisor y del receptor de la información protegida, para intentar contrarrestar esta protección.

Ataque de cumpleaños: Ataque contra las funciones *hash*, consistente en encontrar dos mensajes que den el mismo resumen, en lugar de encontrar un mensaje que dé el mismo resumen que otro determinado, cosa, esta última, que requiere muchas más operaciones.

Ataque de diccionario: Ataque contra los métodos de autenticación de entidad basados en contraseñas consistente en probar las palabras de un diccionario hasta encontrar la correcta.

Ataque de fuerza bruta: Ataque contra las funciones criptográficas consistente en probar todos los posibles valores de la clave hasta encontrar el correcto.

Autenticación: Protección de la información contra falsificaciones.

Autenticación de entidad: Servicio de seguridad que permite confirmar que un participante en una comunicación es auténtico y no se trata de un impostor que está intentando suplantarlo.

Autenticación de mensaje: Servicio de seguridad que permite confirmar que el originador de un mensaje es auténtico y que el mensaje no ha sido creado o modificado por un falsificador.

Autenticación de origen de datos: Nombre con el que se conoce a veces la autenticación de mensaje.

Autoridad de certificación (CA): Entidad que emite certificados de clave pública que sirven para que los usuarios que confían en esta autoridad se convenzan de la autenticidad de las claves públicas.

Autoridad de certificación (CA) raíz: CA que no tiene ninguna otra superior que certifique la autenticidad de su clave pública y que por tanto tiene un certificado firmado por ella misma.

CA: Ver *Autoridad de certificación*.

Cadena de certificados: Lista de certificados, cada uno de los cuales permite verificar la autenticidad de la clave pública de la CA que ha emitido el anterior, hasta llegar al certificado de una CA raíz.

Certificado de clave pública: También conocido como certificado digital, es una estructura de datos que contiene un nombre de usuario y su clave pública y que está firmado

digitalmente por una autoridad de certificación dando fe de esta asociación usuario-clave pública.

Certificado digital: Certificado de clave pública.

Cifrado: Transformación de un texto en claro, mediante un algoritmo que tiene como parámetro una clave, en un texto cifrado ininteligible para quien no conozca la clave de descifrado.

Cifrado en bloque: Transformación criptográfica en que el texto en claro se divide en bloques y se aplica un algoritmo de cifrado a cada uno de estos bloques.

Cifrado en flujo: Transformación criptográfica en que el texto en claro se combina con una secuencia pseudoaleatoria obtenida a partir de la clave.

Clave: Parámetro de un algoritmo de cifrado o de descifrado que permite definir transformaciones criptográficas distintas sin necesidad de cambiar el algoritmo.

Clave de sesión.: Clave simétrica generada *ad hoc* para proteger un determinado intercambio de información y que es conocida por las dos partes utilizando criptografía de clave pública, para que no pueda ser descubierta por un atacante.

Clave privada.: Clave que permite realizar la transformación criptográfica inversa a la que se obtiene con una clave pública y que es computacionalmente inviable obtener a partir de esta última.

Clave pública.: Clave que permite realizar la transformación criptográfica inversa a la que se obtiene con una clave privada y que se puede obtener fácilmente a partir de esta última.

Clave simétrica.: Clave que permite realizar tanto una transformación criptográfica como la transformación inversa, es decir, cifrado y descifrado.

Código de autenticación de mensaje (MAC): Valor calculado a partir de un texto con una clave secreta y que puede ser utilizado por quien conozca la clave para comprobar la autenticidad del mensaje.

Confidencialidad: Protección de la información contra lectura por parte de terceros no autorizados.

Contraseña: Palabra (“*password*”) o cadena de caracteres secreta, de longitud relativamente corta, usada por una entidad para autenticarse.

Criptografía: Estudio de las técnicas matemáticas para anular la protección que proporciona la criptografía.

Criptografía: Estudio de las técnicas matemáticas para proteger la información de modo que no pueda ser interpretada o modificada por partes no autorizadas.

Criptología: Disciplina que engloba la criptografía y el criptoanálisis.

CRL: Ver *Lista de revocación de certificados*.

Descifrado: Transformación inversa al cifrado para obtener el texto en claro a partir del texto cifrado y la clave de descifrado.

Digest: Nombre que se da a veces a un resumen o *hash*.

Firma digital: Valor calculado a partir de un texto con una clave privada y que puede ser comprobado con la correspondiente clave pública, lo cual permite confirmar que solamente lo puede haber generado el poseedor de la clave privada.

Hash: Cadena de bits, de longitud determinada, que se obtiene a partir de una secuencia de bits de longitud arbitraria, como “resumen” de esta secuencia.

Infraestructura de clave pública (PKI): Conjunto de estructuras de datos, procedimientos y agentes que permiten el uso de la criptografía de clave pública.

Lista de revocación de certificados (CRL): Lista de certificados que han dejado de ser válidos antes de la su fecha de caducidad, emitida y firmada por la misma CA que emitió estos certificados.

MAC: Ver *Código de autenticación de mensaje*.

No repudio: Protección de la información contra denegación de autoría por parte de su originador.

Passphrase: Cadena de caracteres secreta, de longitud generalmente mayor que una contraseña, usada por una entidad para autenticarse.

Password: Ver *Contraseña*.

PKI: Ver *Infraestructura de clave pública*.

Resumen: Ver *Hash*.

Reto-respuesta: Método de autenticación de entidad basado en un valor secreto que la entidad a autenticar debe utilizar para calcular una respuesta válida a un reto que le envía el verificador.

Sal: Conjunto de bits aleatorios que se generan *ad hoc* para modificar una clave de cifrado y que permiten que un mismo texto resulte en textos cifrados distintos aunque se cifre con la misma clave.

Seguridad computacional: Seguridad que proporciona una técnica criptográfica cuyo criptoanálisis requeriría una cantidad de recursos computacionales mucho mayor de lo que está al alcance de nadie.

Seguridad incondicional: Seguridad que proporciona una técnica criptográfica que no permite obtener ninguna información sobre el texto en claro, independientemente de la cantidad de recursos disponibles para el criptoanálisis.

Texto cifrado: Resultado de aplicar un cifrado a un texto en claro.

Texto en claro: Información directamente inteligible.

Bibliografía

[1] Menezes, A. J.; van Oorschot, P. C.; Vanstone, S. A. (1996). *Handbook of Applied Cryptography*. Boca Raton: CRC Press.

[2] Stallings, W. (2020). *Cryptography and Network Security, Principles and Practice, 8th ed.* London: Pearson.